

Introduction to Node.js

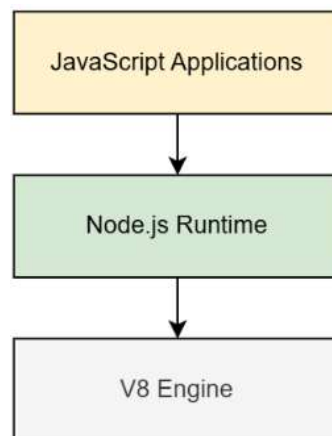
What is Node.js?

Node.js is an open-source, cross-platform runtime environment that allows us to execute JavaScript code outside a web browser. Traditionally, JavaScript was used exclusively in browsers to create interactive web pages. Node.js changed this by enabling JavaScript to be run on servers as well.

At its core, Node.js is built on the V8 JavaScript engine, which powers Google Chrome. V8 compiles JavaScript into machine code, making it fast and efficient. Node.js takes this powerful engine and surrounds it with additional capabilities to perform tasks that browsers don't handle, such as:

- **Handling HTTP requests:** Like responding to web clients (browsers) when they access a website.
- **Interacting with file systems:** Reading from or writing to files on a server.
- **Communicating with databases:** Fetching or storing data for web applications.
- **Building APIs:** Creating interfaces that other programs or services can use.

Essentially, Node.js transforms JavaScript from a browser-only language into a versatile, full-stack development tool that powers both the frontend and backend.



The Node.js stack: JavaScript application, Node.js runtime, and V8 engine

Why use Node.js?

Node.js is especially well-suited for certain types of applications due to its efficiency and scalability:

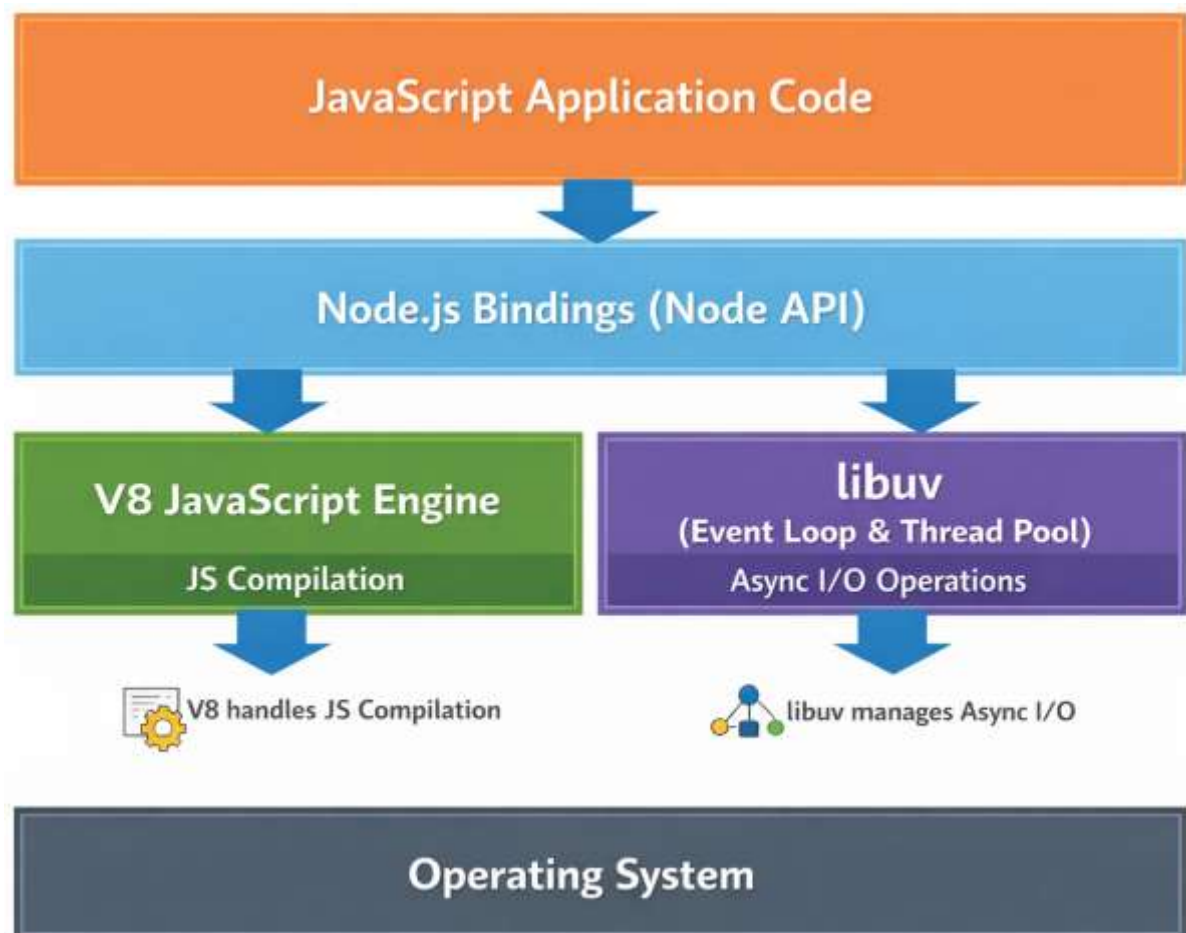
- **Real-time applications:** Node.js is ideal for chat apps or multiplayer games, where handling many simultaneous connections is crucial. Its ability to process multiple requests without waiting for each one to complete allows it to handle real-time data effectively.
- **I/O-heavy applications:** Applications that involve heavy input/output tasks, such as reading/writing files, interacting with databases, or handling network requests, benefit greatly from Node.js's non-blocking nature. It allows multiple tasks to be processed concurrently, making the application more responsive.
- **APIs for single-page applications (SPAs):** Node.js is perfect for developing back-end APIs for SPAs. Since both frontend and backend can be written in JavaScript, it simplifies development and creates a unified environment for developers.

Node.js Architecture Overview

Node.js is built on a unique approach to handling tasks, which makes it both efficient and scalable. At its core, Node.js runs JavaScript on a single thread, meaning it processes all tasks using one main execution line. However, this doesn't mean that it's limited to handling one thing at a time. Node.js achieves high performance through its event loop and asynchronous, non-blocking I/O.

Here's how it works: When Node.js encounters a task that takes time—like reading a file, making a database query, or waiting for a network response—it doesn't stop and wait for the task to complete. Instead, it hands off the time-consuming task to the operating system or an internal thread pool to handle in the background. Once the task is finished, Node.js is notified, and the event loop picks up the result for further processing.

This design allows Node.js to handle many tasks simultaneously without creating multiple threads for each request, making it lightweight and efficient. For applications like web servers or APIs, where many requests involve reading files or querying databases, this architecture enables Node.js to handle thousands of users at once without slowing down.



Node.js Architecture Overview

In synchronous execution, each task must finish before the next one begins, leading to a sequential, blocking flow

By combining JavaScript's simplicity with a powerful event-driven architecture, Node.js provides developers with an efficient way to build scalable and responsive applications.

How V8, libuv & the Node.js APIs Fit Together

1. The Three Core Components

Component	What it is	Language	Responsibility
V8	Google's open-source JavaScript engine (also powers Chrome)	C++	Parses, compiles and executes your JavaScript. Owns the Call Stack, the Heap, and the Microtask Queue.
libuv	A cross-platform C library originally built for Node.js	C	Provides the Event Loop, a Thread Pool, and asynchronous access to the file system, networking, DNS, and timers.
Node.js Bindings	Thin C++ glue code (process.binding / internalBinding)	C++	Exposes libuv's and V8's capabilities to JavaScript as the fs, http, net, etc. modules you actually call.

NOTE: V8 has no concept of files, sockets, or timers on its own — it only knows how to run JavaScript. Every 'async superpower' Node.js has comes from libuv, not from V8.

2. The Layered Architecture

The diagram below shows how a request travels down from your application code to the operating system, and which layer is responsible for what.

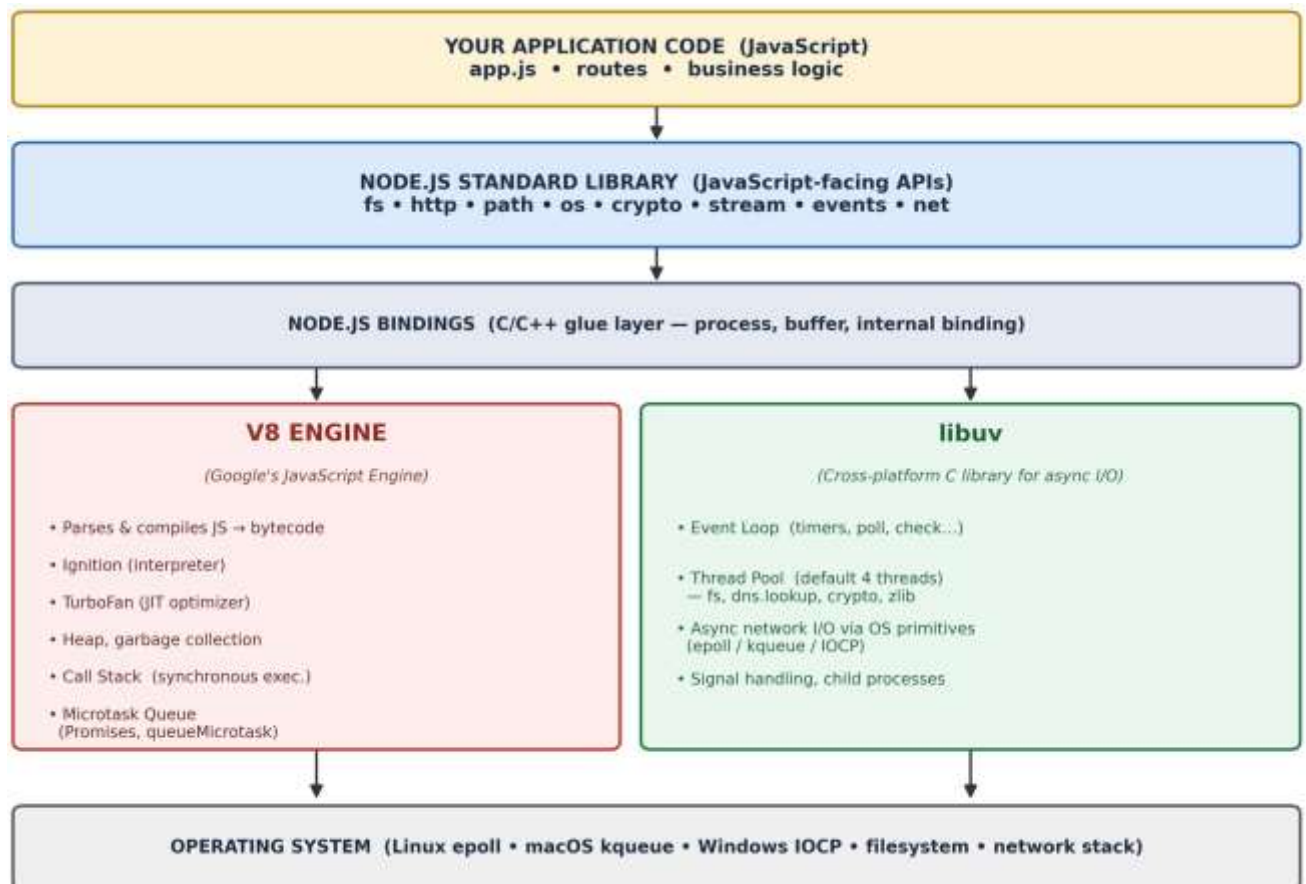


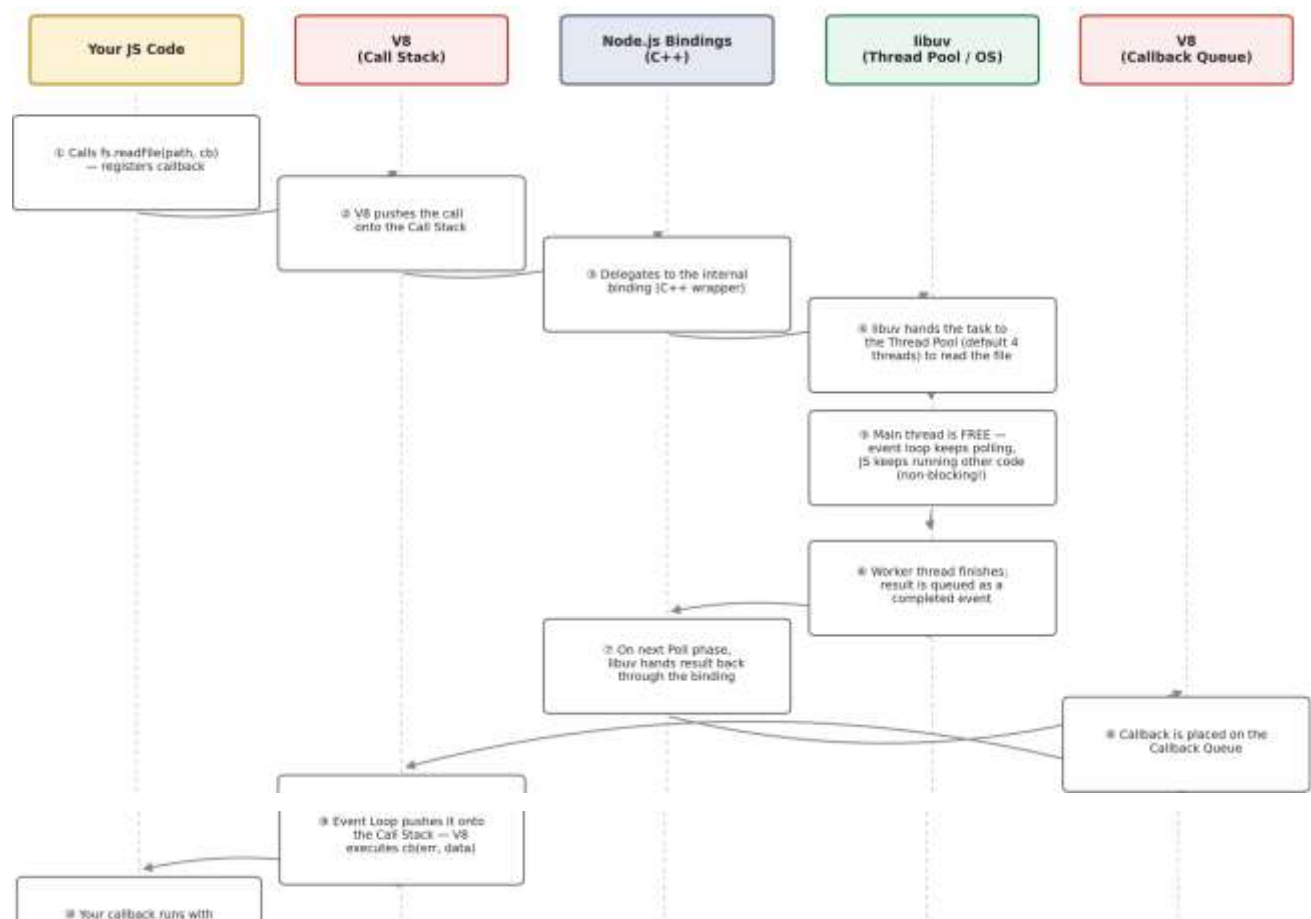
Fig. 1 — Layered view of the Node.js runtime: JS application → Node.js standard library → C++ bindings → V8 + libuv → Operating System.

3. Why Two Engines Instead of One?

- **V8 is deliberately single-purpose:** it is extremely good at parsing and JIT-compiling JavaScript to near-native machine code, but it was designed for a browser's main thread — it has no built-in notion of asynchronous file or network I/O.
- **libuv fills exactly that gap:** it gives Node.js a consistent, cross-platform way to perform I/O without blocking, using whichever mechanism the operating system offers.
- Ryan Dahl (Node.js's creator) chose V8 for raw execution speed and libuv (originally written for Node) to handle everything V8 could not — this separation of concerns is what makes Node.js both fast and non-blocking.

4. Step-by-Step: Life of an Asynchronous Call

Consider a single line of code: `fs.readFile("data.txt", callback)`. Tracing what actually happens makes the relationship between V8, the bindings, and libuv concrete.



Walkthrough

- V8 executes `fs.readFile(...)` on the Call Stack like any normal function call — this part is perfectly synchronous.
- The `fs` module (JavaScript) forwards the request through the internal C++ binding into libuv.
- libuv recognises this as a filesystem operation and hands it to its internal Thread Pool (default size 4), because most operating systems do not offer true async file I/O.

- Because the actual reading happens on a worker thread, V8's Call Stack is immediately free — the main thread moves on to the next line of your script. This is the essence of non-blocking I/O.
- When the worker thread finishes, libuv doesn't call your callback immediately — it queues the completion as an event.
- On its next pass through the Poll phase, the Event Loop picks up that event and hands the callback back to V8.
- V8 places the callback on its Callback Queue, and once the Call Stack is empty, the Event Loop pushes it onto the stack for execution — this is why callbacks always run 'later', never in the middle of your current synchronous code.

Features of the Node.js Runtime

1. Single-Threaded Execution

Node.js executes JavaScript using a **single thread**. At any given time, only one piece of JavaScript code is executed because there is only **one Call Stack**.

Although JavaScript execution is single-threaded, Node.js can still handle **thousands of concurrent client requests** by delegating I/O operations to the Event Loop and libuv instead of creating multiple threads.

Advantages

- No race conditions within JavaScript execution.
- No need for synchronization mechanisms such as locks or mutexes.
- Lower memory consumption compared to multi-threaded servers.
- Simpler programming model.

2. Non-Blocking I/O

One of the biggest strengths of Node.js is its **non-blocking, asynchronous I/O model**.

Whenever Node.js performs a slow operation such as:

- Reading a file
- Querying a database
- Calling another API
- Network communication

it **starts the operation and immediately continues executing other code** instead of waiting.

When the operation completes, the result is returned through:

- Callback
- Promise
- `async/await`

Blocking vs Non-Blocking

Blocking I/O

`fs.readFileSync()`

Waits until file is read

Blocks the Call Stack

Other requests must wait

Non-Blocking I/O

`fs.readFile()`

Immediately returns

Frees the Call Stack

Other requests continue normally

Example: Blocking

```
const fs = require("fs");
const data = fs.readFileSync("sample.txt");
console.log(data.toString());
console.log("Finished");
```

Output:

(File is read)

Finished

Non-Blocking

```
const fs = require("fs");
fs.readFile("sample.txt", (err, data) => {
  console.log(data.toString());
});
console.log("Finished");
```

Output:

Finished

(File content)

Advantages

- Better responsiveness
- High throughput
- Efficient use of CPU
- Handles many simultaneous users

3. Cross-Platform Support

Node.js applications run **without modification** on:

- Linux
- Windows
- macOS

This portability is achieved through **libuv**, which hides operating system differences.

Different operating systems use different asynchronous I/O mechanisms.

Operating System Async Mechanism

Linux	epoll
macOS	kqueue
Windows	IOCP (I/O Completion Ports)

Advantages

- Write once, run anywhere
- No platform-specific code
- Easier deployment across environments

4. Built-in Modules

Node.js comes with a rich **standard library**, eliminating the need for many third-party packages.

Common Built-in Modules

Module	Purpose
fs	File system operations
http	Create HTTP servers
https	Secure HTTP communication
path	File path utilities
os	Operating system information
events	EventEmitter implementation
crypto	Encryption and hashing
stream	Process large data efficiently
util	Utility helper functions
url	URL parsing
buffer	Binary data handling
timers	Timers (setTimeout, setInterval)

Example

```
const os = require("os");  
console.log(os.platform());  
console.log(os.cpus().length);
```

Advantages

- No installation required
- Well-tested and optimized
- Maintained by the Node.js core team

- Reduces dependency on external packages

5. Scalability

Node.js is designed for **building highly scalable applications**.

Instead of creating one thread per client, Node.js uses an **event-driven architecture**, allowing a single process to manage thousands of concurrent connections efficiently.

(a) Event-Driven Concurrency

The Event Loop continuously processes asynchronous events.

Advantages:

- Supports thousands of simultaneous users
- Low memory usage
- Efficient resource utilization

(b) Cluster Module

A single Node.js process uses only one CPU core.

The cluster module creates multiple Node.js processes (usually one per CPU core).

Each process:

- Has its own Event Loop
- Has its own memory
- Shares the same server port

This allows applications to fully utilize multi-core processors.

(c) Worker Threads

CPU-intensive tasks can slow down the main Event Loop.

The worker_threads module creates separate threads for heavy computations such as:

- Image processing
- Machine learning
- Video encoding
- Data analysis

This keeps the main thread responsive.

Runtime Environment: JavaScript Outside the Browser

Node.js is a runtime environment, which means it doesn't come with a graphical user interface like some other development tools. We interact with Node.js through a command-line interface (CLI). Node.js, use its interactive environment (REPL), and create a basic script.

Running Node.js in interactive mode (REPL)

The Node.js REPL lets us interact with JavaScript directly. REPL stands for Read-Eval-Print Loop:

- **Read:** It reads the input.
- **Eval:** It evaluates the input as JavaScript code.

- **Print:** It prints the result of the evaluation.
- **Loop:** It waits for the next command.

Using the REPL is a quick way to test ideas, debug code, or try out JavaScript features. Let's get started.

REPL workflow

To start the Node.js REPL, type `node` in the terminal and press "Enter". This launches an interactive environment where you can type JavaScript code and see the results immediately. The REPL is excellent for testing quick JavaScript snippets without creating a file. However, to save and reuse code, we usually write scripts.

What is Node.js Runtime

The **Node.js runtime** is an execution environment that allows JavaScript to run outside the browser. It combines the **V8 JavaScript engine** (developed by Google for Chrome) with **libuv**, a cross-platform library handling asynchronous I/O operations.

Unlike traditional server environments that spawn new threads per request, Node.js handles thousands of concurrent connections efficiently using its event-driven model.

JAVASCRIPT

```
//Check Node.js version and runtime details
console.log('Node.js Version:', process.version);
console.log('Platform:', process.platform);
console.log('Architecture:', process.arch);
console.log('Process ID:', process.pid);
// Memory usage information
const memUsage = process.memoryUsage();
console.log('Heap Used:', Math.round(memUsage.heapUsed / 1024 / 1024), 'MB');
console.log('Heap Total:', Math.round(memUsage.heapTotal / 1024 / 1024), 'MB');
```

Writing and running our first script

A Node.js script is just a plain JavaScript file, but it can contain commands to interact with the server, file system, and more. Let's walk through creating a basic "Hello, World!" script.

Running a script with the `node` command

To run a script in Node.js, we use the `node` command followed by the name of the script file. For example:

```
node <scriptname>.js
```

This command tells Node.js to load and execute the code inside `scriptname.js`.

Global Objects and Modules

Working with JavaScript in Node.js

When using JavaScript in the browser, we work with the `window` object as the global scope, which includes methods like `alert()` and properties like `location`. However, in Node.js, there is no `window` object because it operates outside the browser environment. Instead, Node.js uses its own global object, `global`, to store globally accessible properties and functions. Functions like `console.log()` and `setTimeout()` are part of this global scope and can be used directly in Node.js applications.

While we can reference properties on the `global` object explicitly, in most cases, we can access global properties directly without needing the `global.` prefix.

Common global properties and functions

Here are a few commonly used properties and functions from the global scope in Node.js:

- `console`: Used for printing output, similar to logging in the browser.
- `setTimeout` and `setInterval`: Functions for scheduling delayed and repeated code execution.
- `process`: Provides information and control over the current Node.js process (we'll cover `process` in more detail later).

These examples illustrate the types of functionality available globally in Node.js, forming part of the tools we'll commonly use when developing applications.

Using `setTimeout` in Node.js

The `setTimeout` function allows us to delay the execution of a function. This can be useful in asynchronous programming when we want to execute code after a certain amount of time.

Using `setTimeout()`:

```
setTimeout(() => {  
  console.log("This message appears after a 1-second delay");  
}, 1000);
```

The function provided to `setTimeout` will run once, after a delay specified in milliseconds.

Using `setInterval` in Node.js

The `setInterval` function is similar but repeatedly executes the function at specified intervals until stopped.

```
setInterval(() => {  
  console.log("This message repeats every second");  
}, 1000);
```

To stop `setInterval`, we can use the `clearInterval` function by storing the interval's ID and passing it to `clearInterval`:

```
const intervalId = setInterval(() => {  
  console.log("This message repeats every second");  
}, 1000);  
clearInterval(intervalId); // Stops the interval when needed
```

Introduction to Modules

In Node.js, a **module** is a reusable block of code that can be imported into other files. Modules are a cornerstone of Node.js, enabling developers to organize applications into smaller, reusable, and maintainable blocks of code. Node.js uses the CommonJS module system, which provides a standard way to define and import modules.

Built-in modules

Node.js comes with several built-in modules that provide core functionality. Examples include:

- **fs**: For interacting with the file system.
- **http**: For creating HTTP servers.
- **path**: For working with file and directory paths.

These modules add essential capabilities to Node.js applications. We'll explore them in more detail later in the course as we build and expand our applications.

Using **require** to import modules

The **require** function lets us import both built-in and custom modules. When we import a module with **require**, Node.js looks for it in the following order:

1. Built-in modules (like **fs** and **http**).
2. Custom modules, typically created within our project.

load the **path** module in the REPL and use one of its methods:

```
const path = require('path');  
console.log(path.basename('/path/to/file.txt')); // Outputs: file.txt
```

Creating a custom module

In Node.js, creating a custom module involves writing code in a separate JavaScript file and exporting it using **module.exports**. By exporting with **module.exports**, we make specific functions or objects available to other files, organizing our code into reusable components.

Step 1: Writing the custom module code

Suppose we want to create a module for basic math operations. We would start by writing our custom code in a file called **mathOperations.js**.

```
1 // mathOperations.js  
2 const add = (a, b) => a + b;  
3 const subtract = (a, b) => a - b;  
4  
5 module.exports = { add, subtract };
```

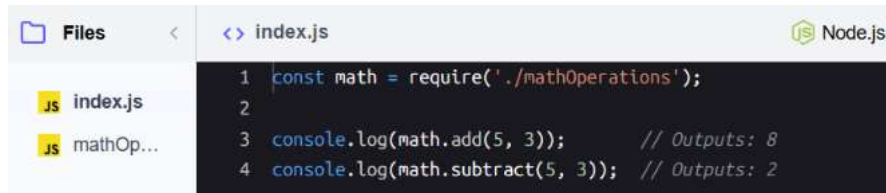
The mathOperations.js file

Explanation:

- **Lines 2–3:** We define two functions: `add` and `subtract`, which perform addition and subtraction respectively.
- **Line 5:** We use `module.exports` to make these functions available to other files.

Step 2: Importing and using the custom module

Once we've created `mathOperations.js`, we can use it in another file by importing it with `require`.



```
1 const math = require('./mathOperations');
2
3 console.log(math.add(5, 3)); // Outputs: 8
4 console.log(math.subtract(5, 3)); // Outputs: 2
```

Importing the mathOperations

Explanation:

- **Line 1:** We use `require('./mathOperations')` to import the `mathOperations` module, giving us access to its functions.
- **Lines 3–4:** We then call `math.add(5, 3)` and `math.subtract(5, 3)` to use the addition and subtraction functions.

This example shows how we can organize code into a separate module and then use it in other parts of our application, keeping our codebase modular and manageable.

Different ways to import and export modules

In Node.js, the `require` function allows us to import modules, and there are different ways to use it depending on our needs. For example, we can import an entire module like this:

```
1 const os = require('os'); // Import the entire os module
2 console.log(os.platform()); // Outputs the platform
3 console.log(os.totalmem()); // Outputs the total memory
```

Alternatively, if we only need specific components from a module, we can extract them directly using destructuring:

```
1 const { platform, totalmem } = require('os'); // Import specific components
2 console.log(platform()); // Outputs the platform
3 console.log(totalmem()); // Outputs the total memory
```

Both approaches are valid. Use the first approach when we need multiple methods or properties from the module, and the second for cleaner, more focused code when we only need specific components.

In some cases, we might see this less common approach where the result of `require` is assigned directly to a variable for immediate use:

```
1 const platform = require('os').platform; // Import only the platform method
2 console.log(platform()); // Outputs the platform
```

While this method works, it is less readable and less idiomatic. It's better to use destructuring or import the whole module for clarity and maintainability.

For modules with a single primary feature, like a custom utility module, it's common to assign `require` directly to a variable representing that feature. For example:

```

1 // utils.js
2 const greet = (name) => `Hello, ${name}!`;
3 module.exports = greet;

```

When importing this module, we can assign it directly to a variable:

```

1 const greet = require('./utils'); // Import the primary export directly
2 console.log(greet('Alice')); // Outputs: Hello, Alice!

```

This approach works well because the `utils` module is designed to export only the `greet` function, making it simple and focused. This pattern is common in Node.js for modules that serve a specific purpose.

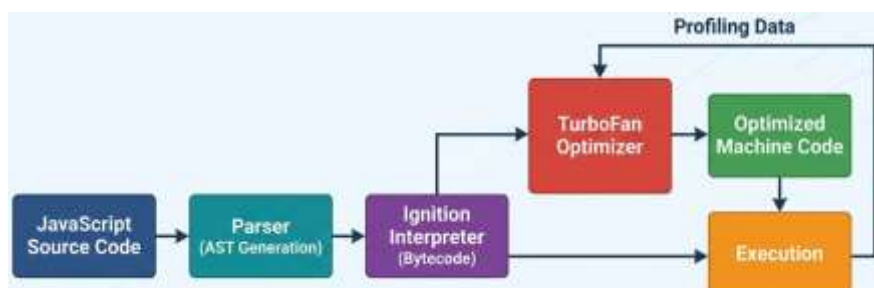
V8 Engine and JavaScript Compilation

The **V8 engine** compiles JavaScript directly to native machine code using **Just-In-Time (JIT) compilation**, making Node.js extremely fast. V8 performs several optimizations:

1. **Parsing** - Converts JS source to Abstract Syntax Tree (AST)
2. **Ignition** - Interprets and generates bytecode
3. **TurboFan** - Optimizes hot code paths to machine code
4. **Garbage Collection** - Automatic memory management

V8's optimization strategies include:

- **Inline caching** for property access
- **Hidden classes** for object shape optimization
- **Dead code elimination** removing unused branches



V8 Compilation Pipeline

libuv And Cross-Platform Abstraction

libuv is the C library that provides Node.js with its asynchronous I/O capabilities. It abstracts operating system differences, ensuring consistent behavior across platforms.

libuv provides:

- **Event loop** implementation
- **Thread pool** (default 4 threads) for blocking operations
- **Async TCP/UDP sockets**
- **File system operations**
- **Child process management**
- **Signal handling**

Operations handled by the thread pool:

- File system operations (fs.readFile, fs.writeFile)
- DNS lookups (dns.lookup)
- Cryptographic operations (crypto.pbkdf2)
- Compression (zlib)

```
// Set thread pool size BEFORE requiring any modules
// Default is 4, maximum is 1024
process.env.UV_THREADPOOL_SIZE = 8;
const crypto = require('crypto');
const fs = require('fs');
// These operations use the thread pool
const start = Date.now();
// Simulate CPU-intensive crypto operations
for (let i = 0; i < 4; i++) {
  crypto.pbkdf2('password', 'salt', 100000, 512, 'sha512', () => {
    console.log(`Hash ${i + 1} completed in ${Date.now() - start}ms`);
  });
}
```

Introduction to Asynchronous Programming

Imagine you're at a coffee shop. You order a latte, but instead of the barista preparing your drink while serving other customers, they stand idle, waiting for your latte to be ready. This would slow everything down, right?

In traditional programming, this is how tasks are often handled: one at a time, in sequence. If a task takes a while, like reading a file or making a network request, everything else waits. This is called **blocking behavior**.

In synchronous execution, each task must finish before the next one begins, leading to a sequential, blocking flow

JavaScript takes a different approach. It uses **asynchronous programming** to stay efficient. When faced with a time-consuming task, JavaScript hands it off to the environment (e.g., Node.js or the browser) and keeps working on other tasks. Once the original task is done, the result is picked up and processed.



In asynchronous execution, tasks can run independently, allowing a non-blocking, parallel flow that maximizes efficiency.

This design makes JavaScript ideal for applications that deal with many requests or heavy I/O operations, such as servers or real-time apps. In this lesson, we'll explore how asynchronous programming works and why it's so powerful.

Example: Synchronous vs. asynchronous execution

Consider two tasks: "Prepare coffee" and "Read an email." In a synchronous model, we'd prepare coffee and wait for it to be done before reading the email. In an asynchronous model, we could start making coffee and then read the email while waiting. Let's see how Node.js approaches this.

```
1 console.log("Task 1: Start preparing coffee");
2
3 * setTimeout(() => {
4   console.log("Task 2: Coffee is ready!");
5 }, 2000);
6
7 console.log("Task 3: Start reading an email");
```

Demonstrating asynchronous execution with `setTimeout` in Node.js

Explanation

- **Line 1:** Logs "Task 1: Start preparing coffee" immediately.
- **Lines 3–5:** The `setTimeout` schedules "Task 2: Coffee is ready!" to run after a 2-second delay, without blocking other code. This function is a callback, meaning it will run after the delay is complete.
- **Line 7:** Logs "Task 3: Start reading an email" immediately, demonstrating that synchronous tasks (like this console log) continue to execute even though an asynchronous task (the `setTimeout` callback) is scheduled.

This example illustrates that even though we scheduled Task 2 to run after a delay, Task 3 executes immediately, demonstrating non-blocking behavior.

Understanding Callbacks

Imagine you're baking cookies. Once they're in the oven, you need to take them out when they're ready to prevent them from burning. Instead of waiting by the oven, you set a timer and use the time to handle other tasks. When the timer goes off, it reminds you to take out the cookies; this is the action that needs to happen *after* the timer completes.

In programming, callbacks are like the action of taking the cookies out. They define what needs to happen when an asynchronous task completes. The program, like the timer, can continue running other tasks in the meantime and execute the callback when it's time to handle the result of the operation. This allows the program to remain efficient and responsive.

What is a callback?

A **callback** is simply a function passed as an argument to another function, intended to be executed once a specific operation is complete. Callbacks are especially useful for handling asynchronous tasks because they allow code to continue running without waiting.

We've already seen callbacks in action with `setTimeout`. Recall how `setTimeout` took a function as an argument and executed it after a delay, without pausing the rest of the code. Let's now see how callbacks work in other scenarios, especially when handling tasks that take time, like reading and writing files.

Callbacks allow us to:

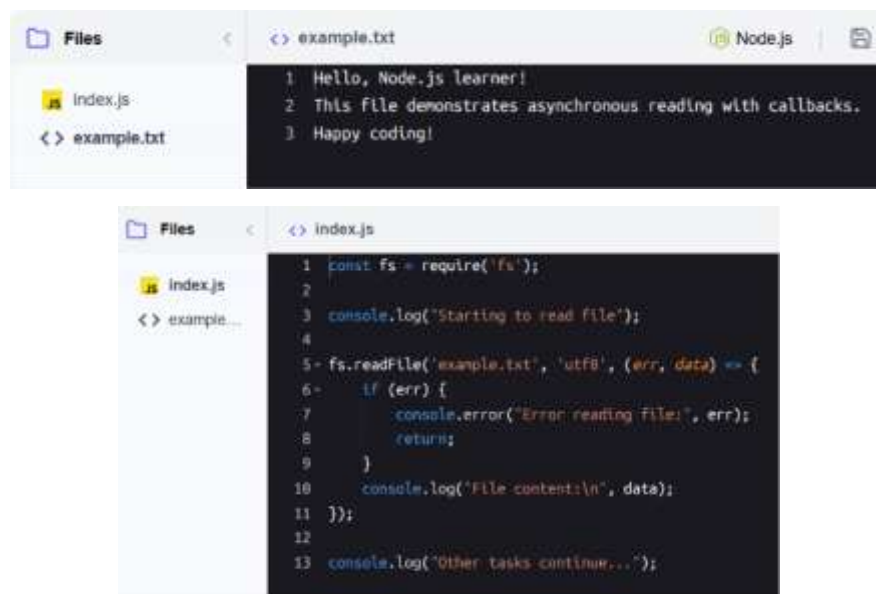
- Run asynchronous tasks without blocking the main thread.
- Respond to events, such as data loading or file operations, only after those tasks complete.
- Improve efficiency by allowing code to continue executing other tasks without waiting.

Example: Reading a file with `fs.readFile`

The `fs.readFile` function lets us read a file asynchronously, allowing other operations to proceed while the file is being loaded. This function accepts three parameters:

- **File path:** The name and path of the file to read.
- **Encoding:** The character encoding for reading the file (e.g., `'utf8'`).
- **Callback function:** A function with two parameters, `error` and `data`, that handles the result of the read operation.

Let's explore how `fs.readFile` uses callbacks to handle file reading asynchronously.



Using a callback with `fs.readFile` to read a file asynchronously

Explanation

- **Line 1:** Imports the `fs` module to enable file system operations.
- **Line 3:** Logs "Starting to read file".
- **Lines 5–11:** Calls `fs.readFile` to read `example.txt`, with a callback function to handle the result:
 - If an error occurs, it logs the error.
 - If the file read is successful, it logs the file content.
- **Line 13:** Logs "Other tasks continue..." immediately, demonstrating that Node.js doesn't wait for the file operation to complete.

This example shows how asynchronous tasks in Node.js can proceed without blocking other operations.

Callbacks and Error-First Convention

Node.js's original async convention, and lists its rules and common pitfalls:

Convention rules:

- First parameter is always the error (or null if success)
- Always check for errors before processing results
- Never throw inside callbacks — pass errors to the callback instead

Common pitfalls:

- Callback hell - Deeply nested callbacks become unreadable
- Forgetting to return after error handling
- Calling callback multiple times accidentally

```
const fs = require('fs');
function processFile(filename, callback) {
  fs.readFile(filename, 'utf8', (err, data) => {
    if (err) {
      // Always return after calling callback with error
      return callback(err, null);
    }
    try {
      const parsed = JSON.parse(data);
      callback(null, parsed);
    } catch (parseError) {
      // Pass parsing errors to callback, don't throw
      callback(parseError, null);
    }
  });
}
// Usage
processFile('data.json', (err, result) => {
  if (err) {
    console.error('Error:', err.message);
    return;
  }
  console.log('Success:', result);
});
```

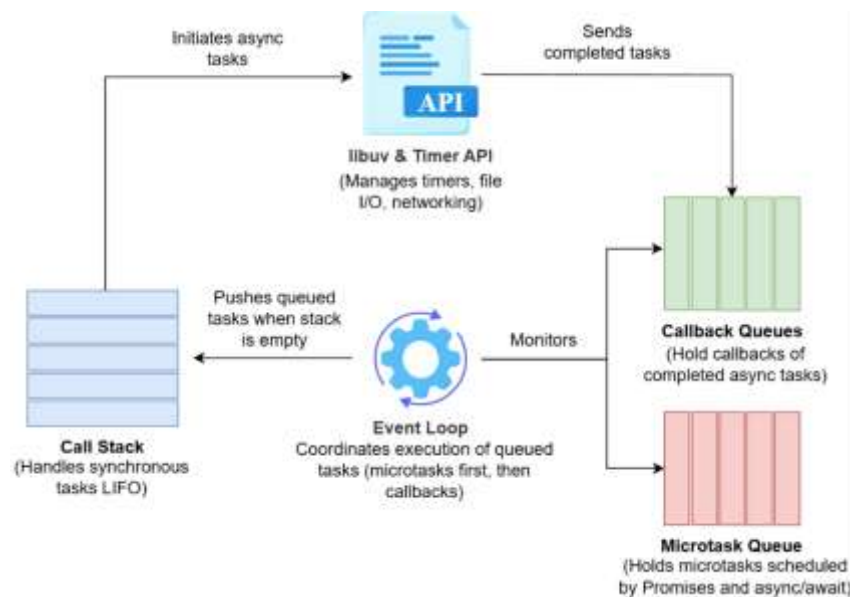
Under the Hood of Asynchronous Execution

Core components of asynchronous execution

When Node.js runs a file with both synchronous and asynchronous code, it coordinates execution using the following four components.:

1. **Call stack:** The primary space where JavaScript code runs, handling synchronous tasks—such as variable declarations or `console.log` statements—immediately in the order they appear in the code. If a piece of code calls a function, that function is added to the top of the call stack. Once the function finishes executing, it is removed from the stack. For asynchronous functions like `setTimeout`, Node.js initially adds them to the call stack, but instead of waiting there, they hand off control to libuv.
2. **libuv and Timer API:** Node.js uses **libuv**, a library that manages various asynchronous operations like timers, file I/O, and networking, using different mechanisms for each type of task. For example:
 - **Timers (`setTimeout`, `setInterval`):** Handled using efficient system timers, allowing the main thread to remain free for other tasks.

- **File I/O operations:** Offloaded to a thread pool to avoid blocking the event loop, since file system operations can be slow.
 - **Networking I/O:** Managed using non-blocking system calls, enabling Node.js to efficiently handle many network connections.
3. **Callback queues:** After asynchronous operations complete, their callback functions are placed into specific queues based on the type of operation:
 - **Timers queue:** For callbacks scheduled by `setTimeout` and `setInterval`.
 - **I/O callbacks queue:** For completed I/O operations.
 - **Check queue:** For callbacks scheduled by `setImmediate`.
 - **Close callbacks queue:** For callbacks related to closed connections.
 4. **Microtask queue:** A queue for tasks scheduled by Promises and `async/await` (we'll explore these in upcoming lessons). Once the call stack is clear, all microtasks are run before any callbacks from the other queues are processed.
 5. **Event loop:** The event loop is a core part of libuv that allows Node.js to perform non-blocking I/O operations. It continuously cycles through different phases, checking if there are callbacks ready to be executed. When the call stack is empty, the event loop first runs all microtasks, and then picks callbacks from the appropriate queues and adds them to the call stack for execution.



Core components of asynchronous execution

This coordination among the four components enables Node.js to handle multiple tasks seamlessly, making it highly effective for I/O-heavy and real-time applications.

The Node.js Event Loop In Depth

The **event loop** is the heart of Node.js's asynchronous programming model. It continuously monitors the call stack and callback queues, executing callbacks when the stack is empty.

Understanding the event loop phases helps you write predictable asynchronous code and avoid common pitfalls like blocking the main thread.

Phase	Queue Type	Callbacks Executed	Example APIs
Timers	Min-Heap	setTimeout, setInterval callbacks	setTimeout(), setInterval()
Pending Callbacks	FIFO Queue	Deferred I/O callbacks	TCP errors, some OS callbacks
Poll	FIFO Queue	I/O callbacks (except close/timers/check)	fs.readFile(), network I/O
Check	FIFO Queue	setImmediate callbacks	setImmediate()
Close Callbacks	FIFO Queue	Close event callbacks	socket.on('close')
Microtasks	FIFO Queue	Promise callbacks, queueMicrotask	Promise.then(), async/await

It also supplies a worked example showing the actual firing order across timers, check, and microtask queues:

```
const fs = require('fs');
// Timers phase
setTimeout(() => console.log('1. setTimeout'), 0);
// Check phase
setImmediate(() => console.log('2. setImmediate'));
// Microtask queue (runs between phases)
Promise.resolve().then(() => console.log('3. Promise.then'));
// nextTick queue (highest priority microtask)
process.nextTick(() => console.log('4. process.nextTick'));
// Synchronous code runs first
console.log('5. Synchronous');

// Output order:
// 5. Synchronous
// 4. process.nextTick
// 3. Promise.then
// 1. setTimeout (or 2, order varies outside I/O)
// 2. setImmediate (or 1)
```

Timers phase — precision caveats

The timers phase executes callbacks scheduled by setTimeout() and setInterval(); Node.js uses a min-heap data structure to efficiently track timer expiration. Important characteristics highlighted by C4:

- Timer thresholds are minimum delays, not guaranteed exact times
- The event loop may delay execution if previous phases take longer
- setTimeout(fn, 0) doesn't mean immediate execution — it means "as soon as possible after current operations"

Technical detail: Node.js coalesces timers with the same expiration time, executing them in the order they were registered.

```
const start = Date.now();

// Schedule timer for 100ms
setTimeout(() => {
  const delay = Date.now() - start;
  console.log(`Timer fired after ${delay}ms (requested 100ms)`);
}, 100);
```

```
// Blocking operation delays the timer
const blockFor = 200; // milliseconds
while (Date.now() - start < blockFor) {
  // Blocking the event loop
}
console.log('Blocking complete');
// Output: Timer fires after ~200ms, not 100ms
```

Poll phase — where Node.js spends most of its time

The poll phase retrieves new I/O events from the operating system and executes their callbacks. Its behavior:

- 1. Calculate timeout - How long to block for I/O
- 2. Process events - Execute callbacks in the poll queue
- 3. Check for timers - If timers are due, loop back to timers phase

The poll phase will block and wait for I/O if the queue is empty and no timers are scheduled, process all callbacks in the queue before moving on, and yield to the check phase if `setImmediate()` callbacks are pending.

Microtasks: `process.nextTick` and Promises

Microtasks execute between event loop phases, giving them higher priority than regular callbacks. Two microtask queues exist: the `process.nextTick` queue (highest priority, processed first) and the Promise microtask queue (processes after `nextTick`). Recursive `process.nextTick()` calls can starve the event loop, preventing I/O callbacks from executing.

Best practices from C4: use `setImmediate()` for deferred execution when possible; reserve `process.nextTick()` for cases requiring immediate async execution; Promises automatically use the microtask queue via `.then()`.

```
// Demonstrating microtask priority
setImmediate(() => console.log('1. setImmediate'));
setTimeout(() => console.log('2. setTimeout', 0));
Promise.resolve()
  .then(() => {
    console.log('3. Promise 1');
    return Promise.resolve();
  })
  .then(() => console.log('4. Promise 2'));
process.nextTick(() => {
  console.log('5. nextTick 1');
  process.nextTick(() => console.log('6. nextTick 2'));
});
console.log('7. Synchronous');

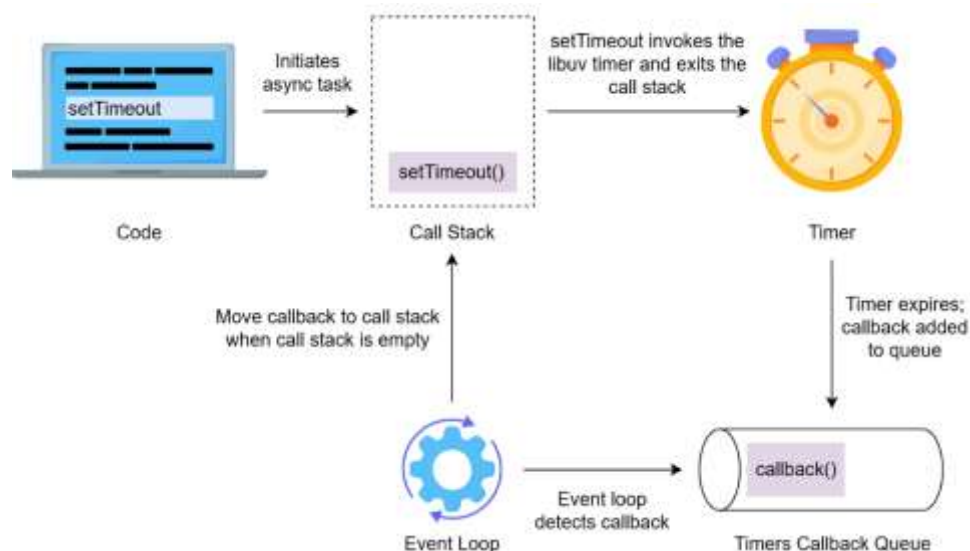
// Output:
// 7. Synchronous
// 5. nextTick 1
```

```
// 6. nextTick 2
// 3. Promise 1
// 4. Promise 2
// 2. setTimeout
// 1. setImmediate
```

Step-by-step flow of `setTimeout`

Here's a closer look at how `setTimeout` operates asynchronously within Node.js, highlighting the steps involved in managing its timing and execution without blocking other tasks:

1. **Call stack initiation:** When `setTimeout` is called, it's initially added to the call stack as a function call.
 - I. **Example:** `setTimeout(() => console.log("Hello"), 2000);`
2. **Timer setup with libuv:** Inside `setTimeout`, a request is made to the Timer API in libuv to start a timer countdown. After initiating the timer, the `setTimeout` function finishes and is removed from the call stack, freeing it up for other tasks.
3. **Timer expiration and callback queuing:** When the timer's delay has elapsed, libuv adds the callback function to the timers queue in the event loop. Note that the callback is added after at least the specified delay, but actual execution might occur slightly later if the event loop is busy or the call stack is not empty.
4. **Event loop execution:** The event loop processes its phases in order. When it reaches the timers phase, it checks the timers queue. If the call stack is empty, the callback is moved from the timers queue to the call stack for execution.



Execution flow of `setTimeout`

Using Promises

As we've seen in previous lessons, Node.js handles asynchronous tasks with callbacks—functions that execute once a task completes. However, as the complexity of asynchronous tasks grows, so does the problem of managing these callbacks. Nesting multiple callbacks quickly leads to **callback hell**, a situation where deeply nested and hard-to-read code becomes challenging to debug and maintain.

For example, imagine a program that reads a file, processes its contents, then writes to another file and logs the result. With callbacks, this can get messy:

```
1 // Example of callback hell
2 fs.readFile('input.txt', 'utf8', (err, data) => {
3   if (err) return console.error(err);
4   processData(data, (err, result) => {
5     if (err) return console.error(err);
6     fs.writeFile('output.txt', result, (err) => {
7       if (err) return console.error(err);
8       console.log('File written successfully');
9     });
10  });
11 });
```

Example of callback hell in handling asynchronous tasks

This structure makes the flow difficult to follow, increasing the likelihood of errors. Promises offer a way to manage asynchronous code more effectively, making it easier to read and handle errors.

What is a Promise?

A **Promise** represents an asynchronous operation that will complete in the future, either successfully (fulfilled) or with an error (rejected). By allowing us to manage asynchronous code as promises of future results, Promises create a smoother code flow.

Promises provide a way to escape the complexity of nested callbacks by enabling sequential handling of asynchronous tasks. With Promises, error handling becomes centralized and more manageable, making asynchronous code easier to read, debug, and maintain.

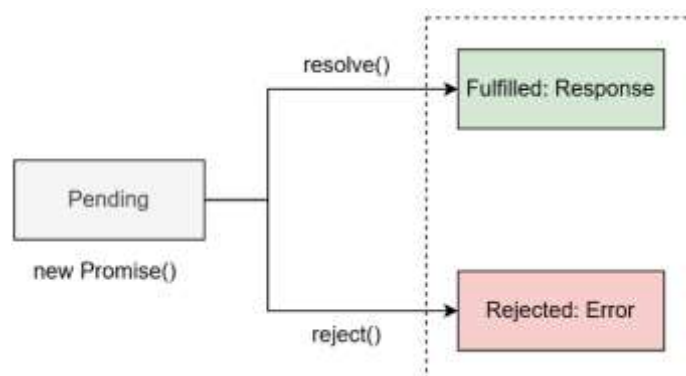
A Promise has three possible states:

1. **Pending:** The initial state, before the asynchronous task completes.
2. **Fulfilled:** The task has completed successfully.
3. **Rejected:** An error occurred in the task.

Creating a Promise

Creating a new Promise involves using `new Promise((resolve, reject) => {...})`, where `resolve` and `reject` are functions:

- `resolve` is called when the task completes successfully.
- `reject` is called when the task encounters an error.



Anatomy of a Promise

To see how Promises work in practice, let's create a simple Promise that simulates a task and handles success or failure. This example will demonstrate how to set up a Promise and manage its outcome with resolve and reject.

```
<> index.js
1- const myTask = new Promise((resolve, reject) => {
2-   // Simulate a task
3-   const success = true; // Change to false to simulate an error
4-
5-   if (success) {
6-     resolve("Task completed successfully!");
7-   } else {
8-     reject("Task failed.");
9-   }
10- });
```

Creating a simple Promise to handle success and failure scenarios

Explanation

- **Line 1:** Defines a new Promise called `myTask`. Inside, two functions, `resolve` and `reject`, determine how to handle success and error cases.
- **Line 3:** Sets a flag, `success`, to simulate whether the task will succeed. Changing this to `false` simulates an error.
- **Lines 5–6:** If `success` is `true`, `resolve` is called with a success message, allowing any handler specified in `.then(...)` to be executed later.
- **Lines 7–9:** If `success` is `false`, `reject` is called with an error message, which will be handled by `.catch(...)`.

Handling Promises with `.then` and `.catch`

After creating a Promise, we can specify how to handle its outcome using `.then(...)` and `.catch(...)`:

- **`.then(...)`:** Defines what happens when `resolve` is called, handling the successful outcome.
- **`.catch(...)`:** Defines what happens when `reject` is called, handling any error.

Let's continue with the previous example to see how to use them.

```
<> index.js
1- const myTask = new Promise((resolve, reject) => {
2-   // Simulate a task
3-   const success = true; // Change to false to simulate an error
4-
5-   if (success) {
6-     resolve("Task completed successfully!");
7-   } else {
8-     reject("Task failed.");
9-   }
10- });
11
12 myTask
13- .then((message) => {
14-   console.log(message); // Logs the success message from resolve
15- })
16- .catch((error) => {
17-   console.error(error); // Logs the error message from reject
18- });
```

Handling Promises with `.then` and `.catch`

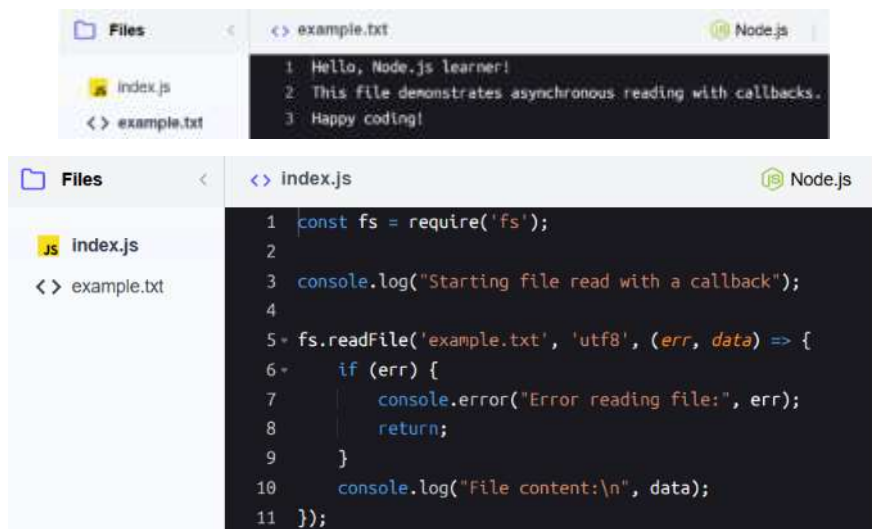
Explanation

- **Line 12:** Invokes `myTask` and specifies how to handle the Promise's result with `.then(...)` and `.catch(...)`.
- **Lines 13–15:** If the Promise resolves successfully, the message passed to `resolve` is logged in `.then(...)`.
- **Lines 16–18:** If the Promise encounters an error, `.catch(...)` logs the error message from `reject`.

Note: Internally, promises work with the event loop similarly to callbacks: once a promise resolves or rejects, its result (or error) is placed in the appropriate callback queue.

Example: Converting a callback to a Promise

Here's an example of reading a file with a callback.



The screenshot shows two panels in VS Code. The top panel, titled 'example.txt', displays the content of a text file: '1. Hello, Node.js learner!', '2. This file demonstrates asynchronous reading with callbacks.', and '3. Happy coding!'. The bottom panel, titled 'index.js', shows the following JavaScript code:

```
1 const fs = require('fs');
2
3 console.log("Starting file read with a callback");
4
5 fs.readFile('example.txt', 'utf8', (err, data) => {
6   if (err) {
7     console.error("Error reading file:", err);
8     return;
9   }
10  console.log("File content:\n", data);
11 });
```

Reading a file with a callback

Explanation

- **Line 5:** Begins reading a file asynchronously, with a callback to handle the result.
- **Lines 6–9:** If there's an error, it logs the error.
- **Line 10:** If successful, it logs the file content.

Now, let's convert this to use Promises.



The screenshot shows the 'index.js' file in VS Code with the following JavaScript code:

```
1 // Using fs.promises to access Promise-based file operations
2 const fs = require('fs').promises;
3
4 console.log("Starting file read with Promises");
5
6 fs.readFile('example.txt', 'utf8')
7   .then(data => {
8     console.log("File content:\n", data);
9   })
10  .catch(err => {
11    console.error("Error reading file:", err);
12  });
```

Reading a file using Promises

Explanation

- **Line 2:** The `fs.promises` import provides access to Promise-based file methods, like `readFile` and `writeFile`, allowing us to handle asynchronous file operations

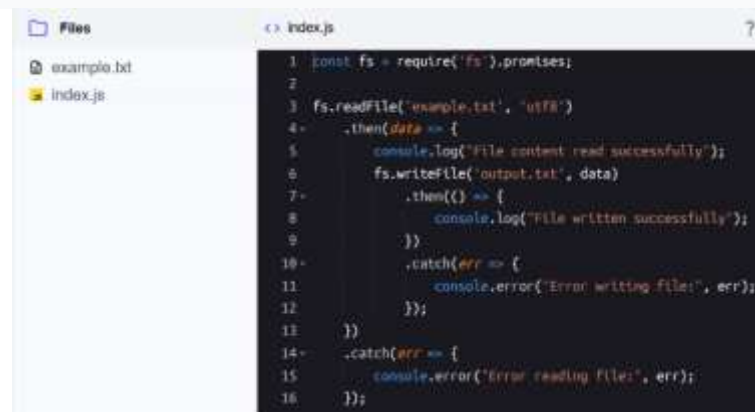
with `.then()` and `.catch()`. This differs from the standard `fs` module, which uses callback-based functions and does not support chaining with Promises.

- **Line 6:** Begins reading the file and returns a Promise. Instead of a callback, we chain `.then()` to handle success and `.catch()` for errors.

Chaining Promises

When working with multiple asynchronous operations, Promises allow us to chain tasks together. However, how we structure the chain can significantly affect code readability and maintainability. Let's compare two approaches to chaining Promises: a nested version and a cleaner, sequential version. Here's an example of handling two tasks—reading a file and writing its content to another file—using nested `.then()` calls:

Note: After executing the code, you can verify the contents of `output.txt` using the `cat output.txt` command.



```
1 const fs = require('fs').promises;
2
3 fs.readFile('example.txt', 'utf8')
4   .then(data => {
5     console.log('File content read successfully');
6     fs.writeFile('output.txt', data)
7       .then(() => {
8         console.log('File written successfully');
9       })
10      .catch(err => {
11        console.error('Error writing file:', err);
12      })
13   })
14   .catch(err => {
15     console.error('Error reading file:', err);
16   });
```

Promise chaining example with file reading and writing

Explanation

- **Line 3:** Initiates the first operation: reading the file `example.txt`.
- **Lines 7–9:** Inside the first `.then()` block, it nests another `.then()` call for writing the content to `output.txt`.
- **Lines 10–12:** Adds a separate `.catch()` block to handle errors for the write operation.
- **Lines 14–16:** The outer `.catch()` block handles errors from the read operation.

To improve readability and maintainability, we can chain `.then()` calls sequentially, using a single `.catch()` to handle errors for the entire chain:



```
1 const fs = require('fs').promises;
2
3 fs.readFile('example.txt', 'utf8')
4   .then(data => {
5     console.log('File content read successfully');
6     return fs.writeFile('output.txt', data);
7   })
8   .then(() => {
9     console.log('File written successfully');
10  })
11  .catch(err => {
12    console.error('Error:', err);
13  });
```

Handling multiple file reads with sequential `.then()` calls

Explanation

- **Line 3:** Starts the first asynchronous task: reading `example.txt`.
- **Line 4–6:** Processes the resolved content (`data`) and returns a new Promise from `fs.writeFile`. This ensures the second task is part of the same chain.
- **Line 7–8:** Handles the resolution of the second task, logging a success message when the file is written.
- **Line 9–11:** A single `.catch()` block handles errors for both the read and write operations.

Promise combinators — `Promise.all`, `allSettled`, `race`, `any` (new topic)

This topic does not appear in the original Nodejs.pdf. The C4 refcard introduces four static Promise methods for running multiple async operations together:

- `Promise.all([...])` - Resolves when ALL succeed, rejects on ANY failure
- `Promise.allSettled([...])` - Waits for all, returns status of each
- `Promise.race([...])` - Resolves/rejects with first settled promise
- `Promise.any([...])` - Resolves with first success, rejects if ALL fail

```
const fetchUser = (id) =>
  new Promise((resolve) =>
    setTimeout(() => resolve({ id, name: `User${id}` }), 100 * id)
  );
const fetchWithError = (id) =>
  new Promise((_, reject) =>
    setTimeout(() => reject(new Error(`Failed ${id}`)), 50)
  );
// Promise.all - All must succeed
Promise.all([fetchUser(1), fetchUser(2), fetchUser(3)])
  .then(users => console.log('All users:', users))
  .catch(err => console.error('One failed:', err));
// Promise.allSettled - Get all results regardless of success/failure
Promise.allSettled([fetchUser(1), fetchWithError(2), fetchUser(3)])
  .then(results => {
    results.forEach((result, i) => {
      if (result.status === 'fulfilled') {
        console.log(`Success ${i}:`, result.value);
      } else {
        console.log(`Failed ${i}:`, result.reason.message);
      }
    });
  });
// Promise.race - First to settle wins
Promise.race([fetchUser(3), fetchUser(1)])
  .then(first => console.log('First result:', first));
```

Async/Await Syntax

In asynchronous programming, efficiently managing multiple tasks is essential for creating responsive and maintainable applications. While Promises provide a structured way to handle asynchronous operations, the `async` and `await` syntax simplifies this further.

Understanding `async` and `await`

The `async` and `await` keywords allow us to work with asynchronous code in a sequential way, making it easier to read and maintain. Here's how each works:

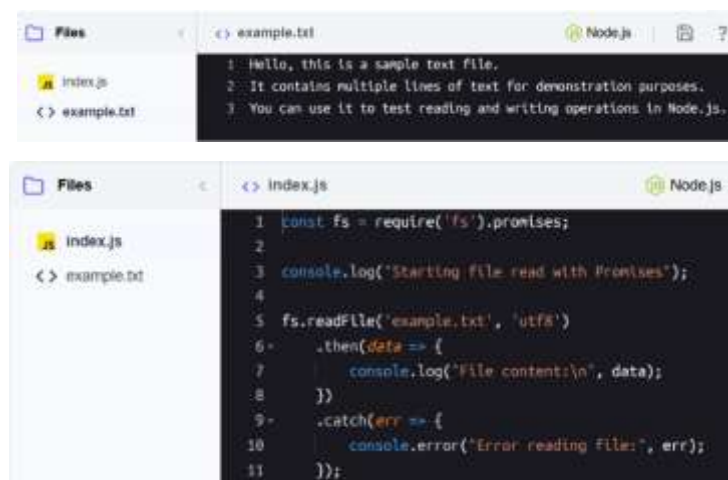
- **`async`:** Declaring a function as `async` automatically returns a Promise. If the function returns a value, `async` wraps it in a Promise.
- **`await`:** Using `await` before a Promise inside an `async` function pauses execution until the awaited Promise resolves, allowing us to handle asynchronous operations in a synchronous-looking flow.

Example: Converting Promises to `async/await`

To illustrate, let's first look at a Promise-based example and then convert it to use `async` and `await` for a cleaner syntax.

Original code with Promises

Here is the code written using Promises:



```
1 Hello, this is a sample text file.
2 It contains multiple lines of text for demonstration purposes.
3 You can use it to test reading and writing operations in Node.js.

1 const fs = require('fs').promises;
2
3 console.log('Starting file read with Promises');
4
5 fs.readFile('example.txt', 'utf8')
6   .then(data => {
7     console.log('File contents:\n', data);
8   })
9   .catch(err => {
10    console.error('Error reading file:', err);
11  });
```

Reading a file asynchronously using Promises

Explanation:

- This code uses `.then()` to handle success and `.catch()` to handle errors.

Refactored code with `async/await`

Let's refactor the code above to implement `async/await` for improved readability:



```
1 const fs = require('fs').promises;
2
3 async function readFileAsync() {
4   try {
5     const data = await fs.readFile('example.txt', 'utf8');
6     console.log('File contents:\n', data);
7   } catch (err) {
8     console.error('Error reading file:', err);
9   }
10 }
11
12 readFileAsync();
```

Reading a file with `async/await` syntax

Explanation

- **Line 3:** Declares `readFileAsync` as an `async` function, which means it returns a Promise.
- **Line 5:** Uses `await` to pause execution until `fs.readFile` completes, storing the result in `data`.

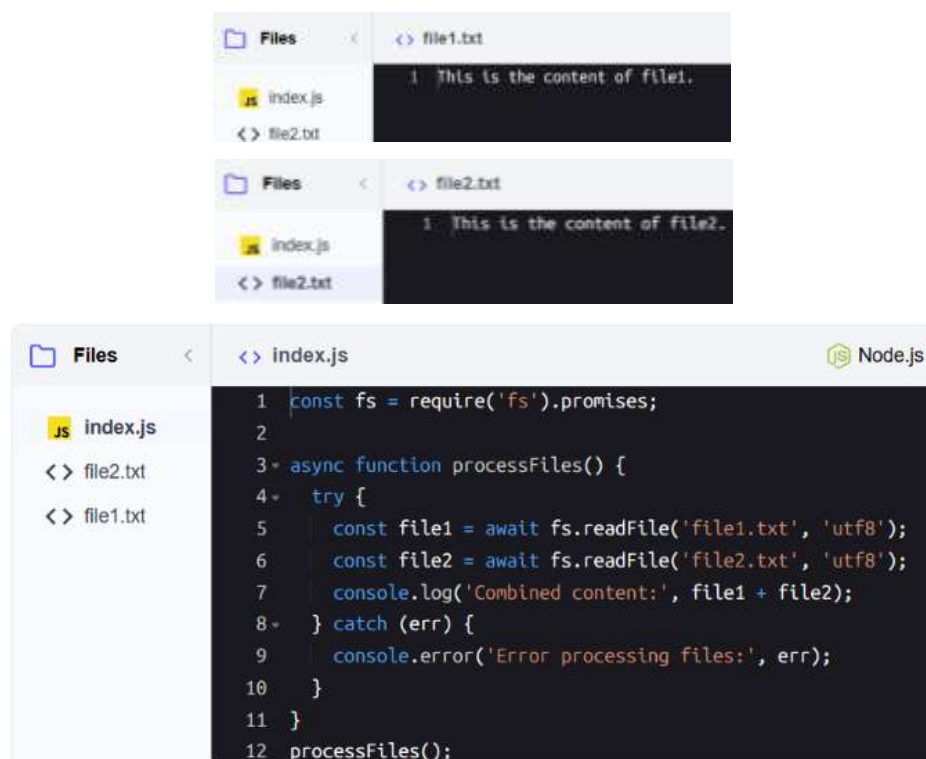
- **Line 6:** Logs the file content if the read operation succeeds.
- **Line 7–9:** Catches and logs any error if `fs.readFile` fails, using `try` and `catch`.

The `try...catch` block in `async/await` syntax is a direct replacement for `.then` and `.catch` in Promises, translating their behavior into a cleaner, synchronous-like structure.

- `.then()`: The code inside `try` replaces the `.then()` block. It handles the resolved value of the Promise. In our example, `await fs.readFile` pauses execution and retrieves the resolved value directly into `data`.
- `.catch()`: The `catch` block replaces `.catch()`, capturing any errors that occur during the execution of the `try` block. This eliminates the need for separate chaining.

Simplifying Promise chaining with `async/await`

`async/await` simplifies handling chained Promises by replacing `.then()` calls with sequential `await` statements. Let's look at an example to see how this works in practice.



The image shows a VS Code editor with three panels. The top panel shows `file1.txt` with the content: `1 This is the content of file1.` The middle panel shows `file2.txt` with the content: `1 This is the content of file2.` The bottom panel shows `index.js` with the following code:

```

1 const fs = require('fs').promises;
2
3 async function processFiles() {
4   try {
5     const file1 = await fs.readFile('file1.txt', 'utf8');
6     const file2 = await fs.readFile('file2.txt', 'utf8');
7     console.log('Combined content:', file1 + file2);
8   } catch (err) {
9     console.error('Error processing files:', err);
10  }
11 }
12 processFiles();

```

Reading multiple files sequentially with `async/await`

Explanation:

- **Line 3:** Declares `processFiles` as an `async` function, enabling the use of `await` inside its body.
- **Line 5:** Uses `await` to pause execution until the Promise from `fs.readFile` resolves for `file1`. The file content is stored in `file1`.
- **Line 6:** Similar to line 3, pauses execution to retrieve the content of `file2`.
- **Line 7:** Logs the combined content of `file1` and `file2`.
- **Lines 8–10:** The `catch` block handles any error that occurs in the `try` block, such as issues with reading files.

This is equivalent to chaining the operations using `.then()` statements.

Introduction to Event-Driven Programming

Imagine a busy restaurant with a skilled staff efficiently handling multiple orders at the same time. As customers place orders, the kitchen gets notified, prepares meals, and updates the staff when an order is ready. This kind of coordination ensures smooth operation even during rush hours.

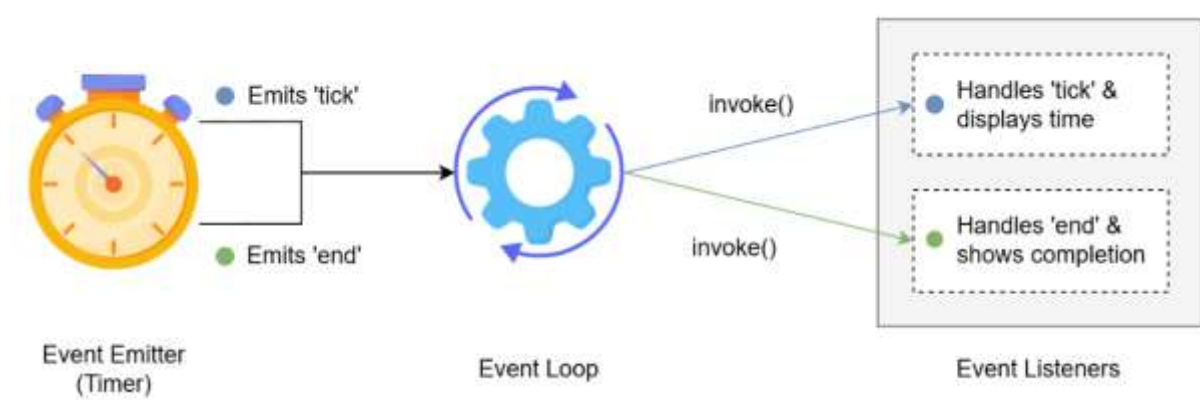
Event-driven programming works in a similar way. Instead of waiting for each task to finish one by one, programs listen for events and respond as they occur. This approach is at the core of Node.js and allows it to handle many tasks simultaneously, like managing user interactions, incoming requests, or file operations, without being overwhelmed.

Key components of event-driven programming

Event-driven programming relies on several key components that enable asynchronous and dynamic behavior in applications.

1. **Event loop:** A mechanism that monitors the program for events and delegates tasks for processing.
2. **Event listeners:** Functions that *listen* for specific events and execute when those events occur.
3. **Event emitters:** Objects that *emit* events, signaling that something has happened, which listeners can then handle.

For instance, consider a countdown timer implemented in Node.js. The timer emits a 'tick' event every second, with listeners responding by displaying the remaining time. When the countdown reaches zero, an 'end' event is emitted, and the listeners respond by indicating the countdown is complete. This example demonstrates how event emitters and listeners work together, with the event loop managing the timely execution of events and their associated callbacks.



EventEmitter emits 'tick' and 'end' events, which the Event Loop routes to their respective listeners for handling

The **EventEmitter** class

Node.js uses the **EventEmitter** class from the **events** module to implement its event-driven architecture. This class enables:

- Emitting custom events to signal that an action has occurred.
- Registering listeners to react to those events.

The `EventEmitter` class provides two key methods for working with events:

1. `.on(eventName, listener)`: Registers a callback function (listener) that runs whenever the specified event is emitted.
2. `.emit(eventName, ...args)`: Triggers the specified event and optionally passes arguments to its registered listeners. Returns `false` if no listeners are registered for the specified event.

These methods make it simple to implement reactive, event-driven logic in our Node.js applications. Here's a basic example to illustrate event-driven programming:

```
<> index.js
1 const EventEmitter = require('events');
2
3 // Create an instance of EventEmitter
4 const myEmitter = new EventEmitter();
5
6 // Add an event listener
7 myEmitter.on('greet', (name) => {
8   console.log(`Hello, ${name}!`);
9 });
10
11 // Emit the event
12 myEmitter.emit('greet', 'Alice');
```

Using EventEmitter to emit and handle a custom "greet" event

Explanation

1. **Line 1:** We import the `events` module, which provides the `EventEmitter` class.
2. **Lines 7–9:** We create an event listener for a custom "greet" event using a callback function, which logs a personalized greeting.
3. **Line 12:** We `emit` the "greet" event, triggering the listener and passing "Alice" as a parameter.

This simple example demonstrates the core idea: events (like "greet") are emitted, and listeners react to them.

Managing Multiple Listeners

In many real-world scenarios, a single event often needs to trigger multiple actions to ensure everything works seamlessly. For example, consider an online shopping platform where a customer places an order. This single event—an order being placed—can set off a chain of actions: logging the order details for record-keeping, sending a confirmation email to the customer, notifying the warehouse to prepare for shipment, and updating the inventory system to reflect the stock change. Each of these tasks is crucial, and they all originate from the same event. This highlights the importance of managing multiple listeners effectively in an event-driven system.

Managing multiple listeners and passing data

The `EventEmitter` class allows attaching multiple listeners to the same event. Each listener can perform a unique task and can accept a different number of parameters, depending on its specific purpose.

When the event is emitted, all listeners are executed in the order they were registered, making it easy to create modular, flexible, and reusable code.

Here's an example of a chat application, where multiple listeners handle the same event with different numbers of parameters.

```
<> index.js
1  const EventEmitter = require('events');
2
3  // Create an EventEmitter instance
4  const chat = new EventEmitter();
5
6  // Listener 1: Log message details
7  chat.on('message', (user, msg) => {
8    console.log(`[To Everyone]: ${user} says: ${msg}`);
9  });
10
11 // Listener 2: Acknowledge message receipt
12 chat.on('message', (user) => {
13   console.log(`Message received from ${user}`);
14 });
15
16 // Emit the 'message' event
17 chat.emit('message', 'Alice', 'Hello, everyone!');
```

Demonstrating multiple listeners for a single event with EventEmitter

Explanation

- **Line 1:** Imports the `events` module.
- **Line 4:** Creates an `EventEmitter` instance named `chat`.
- **Lines 7–9:** The first listener for the `message` event logs the details of the message, including the user who sent it and the message content. This provides a broadcast-style log for all users.
- **Lines 12–14:** The second listener for the `message` event acknowledges the receipt of the message by logging the sender's name.
- **Line 17:** Emits the `message` event, passing user and message data as arguments. Both listeners process the event independently.

Practical tips for managing listeners

Here are some essential tips to help us effectively manage multiple listeners and ensure smooth event handling in our application:

- **Naming conventions:** Use clear and consistent event names (e.g., `join`, `leave`, `message`) to improve readability and maintainability.
- **Order of execution:** Remember that listeners are executed in the order they are registered.
- **Avoid duplication:** Prevent redundant behavior by ensuring the same listener isn't registered multiple times. Debug using `emitter.listeners(event)` to check active listeners.

- **Keep listeners focused:** Assign each listener a clear, single responsibility. For example, a `join` listener might only handle logging, while a `message` listener might handle broadcasting. This separation improves code readability and maintainability.
- **Error handling:** Always listen for the `error` event to handle unexpected issues gracefully.

```
chat.on('error', (err) => {
  console.error(`Error: ${err.message}`);
});
```

Dynamic Event Handling

In real-world applications, events and their listeners often need to change dynamically. For instance:

- Some listeners should only react to an event once.
- Specific listeners might need to be removed based on conditions.
- We may need to debug or track the number of active listeners.
- Listener limits might need to be increased or removed for flexibility.

In this lesson, we'll explore these dynamic features of the `EventEmitter` class with practical examples, continuing with the context of a chatroom application.

Using one-time listeners

The `.once()` method registers a listener that executes only the first time an event is emitted. After that, it's automatically removed.

Let's add a one-time listener to display a welcome message the first time a user joins the chat.

```
<> index.js
1  const EventEmitter = require('events');
2
3  // Create an EventEmitter instance
4  const chat = new EventEmitter();
5
6  // One-time listener for the first "join"
7  chat.once('join', () => {
8    console.log('Welcome to the chat! You are the first one here.');
```

Using `.once()` to display a welcome message for the first user who joins

Explanation

- **Line 1:** Import the `events` module to access the `EventEmitter` class.
- **Line 4:** Create an `EventEmitter` instance named `chat`.

- **Lines 7–9:** Use `.once()` to register a one-time listener for the `join` event. This listener logs a welcome message only when the first user joins.
- **Line 12–14:** Use `.on()` to register a persistent listener for the `join` event, which logs the name of each user joining the chat.
- **Lines 17–18:** Emit the `join` event twice with different users. The welcome message is displayed only once, while the user join messages are logged for each event.

Removing listeners dynamically

The `.removeListener()` method allows removing specific listeners from an event. This can be useful when a listener is no longer relevant.

Let's remove a chatroom listener that tracks message activity after it logs a certain number of messages.

```
<> index.js
1  const EventEmitter = require('events');
2
3  // Create an EventEmitter instance
4  const chat = new EventEmitter();
5
6  function messageLogger(sender, msg) {
7    console.log(`[Logger]: ${sender} says: ${msg}`);
8  }
9
10 // Add a listener for "message"
11 chat.on('message', messageLogger);
12
13 // Emit "message" events
14 chat.emit('message', 'Alice', 'Hello!');
15 chat.emit('message', 'Bob', 'Hi, Alice!');
16
17 // Remove the listener
18 chat.removeListener('message', messageLogger);
19
20 // Emit another "message" event
21 chat.emit('message', 'Charlie', 'Hey everyone!');
```

Removing a specific listener for the message event

Explanation

- **Line 1:** Import the `events` module to access the `EventEmitter` class.
- **Line 4:** Create an `EventEmitter` instance named `chat`.
- **Lines 6–8:** Define a `messageLogger` function to handle the `message` event by logging chat messages.
- **Line 11:** Add the `messageLogger` function as a listener for the `message` event.
- **Lines 14–15:** Emit the `message` event twice. Both times, the `messageLogger` listener is triggered.
- **Line 18:** Remove the `messageLogger` listener from the `message` event using `.removeListener()`.
- **Line 21:** Emit the `message` event again. This time, no output is generated as the listener has been removed.

Counting active listeners

The `.listenerCount()` method (used as `EventEmitter.listenerCount()`) provides the number of active listeners for a specific event. This is especially helpful for debugging.

```
<> index.js
1 const EventEmitter = require('events');
2
3 // Create an EventEmitter instance
4 const chat = new EventEmitter();
5
6 function messageLogger(sender, msg) {
7   console.log(`[Logger]: ${sender} says: ${msg}`);
8 }
9
10 chat.on('message', messageLogger);
11 console.log(`Active listeners for "message":
12 ${EventEmitter.listenerCount(chat, 'message')}`);
13
14 chat.removeListener('message', messageLogger);
15 console.log(`Active listeners for "message":
16 ${EventEmitter.listenerCount(chat, 'message')}`);
```

Counting active listeners for an event

Explanation

- **Line 10:** Add the `messageLogger` listener to the message event.
- **Lines 11–12:** Use `EventEmitter.listenerCount()` to log the count of active listeners for the `message` event.
- **Line 14:** Remove the `messageLogger` listener using `.removeListener()`.
- **Lines 15–16:** Log the updated count of active listeners for the `message` event. It should now be zero.

Managing listener limits

The `EventEmitter` class in Node.js has a default listener limit of 10 listeners per event. This limit exists to help developers identify potential memory leaks in their applications. If the number of listeners attached to a specific event exceeds this limit, Node.js will display a `MaxListenersExceededWarning` in the console, but it does not prevent additional listeners from being registered. This warning is logged only the first time the limit is exceeded for a particular event type, ensuring the console is not flooded with repetitive warnings.

This default limit is not strict; we can adjust it to suit the needs of our application using the `.setMaxListeners()` method:

- **Increase the limit:** Use `.setMaxListeners(n)` to allow more listeners for a specific event.
- **Remove the limit:** Use `.setMaxListeners(-1)` to remove the listener limit entirely. However, this should be used cautiously to avoid unintentional memory leaks.

```

<> index.js
1  const EventEmitter = require('events');
2
3  // Create an EventEmitter instance
4  const chat = new EventEmitter();
5
6  // Adjust listener limit
7  chat.setMaxListeners(2); // Set a custom limit
8
9  // Handle warnings
10 process.on('warning', (warning) => {
11     if (warning.name === 'MaxListenersExceededWarning') {
12         console.warn('Warning: ${warning.message}');
13     }
14 });
15
16 // Add listeners in a loop to exceed the limit
17 for (let i = 1; i <= 4; i++) {
18     chat.on('message', () => {
19         console.log('Listener ${i} added');
20     });
21 }
22
23 // Emit an event to test the listeners
24 chat.emit('message');

```

Demonstrating EventEmitter's listener limit and handling MaxListenersExceededWarning

Explanation:

- **Line 1:** Import the `events` module to access the `EventEmitter` class.
- **Line 4:** Create an instance of `EventEmitter` named `chat` to manage event listeners.
- **Line 7:** Set a custom listener limit of `2` using the `setMaxListeners()` method.
- **Line 10–14:** Add a global `process.on('warning')` handler to capture and log any `MaxListenersExceededWarning` warnings. Note that the `process` object is a global Node.js object that we'll cover later in the course.
- **Line 17–20:** Use a `for` loop to add `4` listeners to the `message` event, intentionally exceeding the limit of `2`.
- **Line 23:** Emit the `message` event to trigger all active listeners for this event.

By carefully managing listener limits and monitoring warnings, we can ensure our event-driven application runs efficiently and safely.

Core Node.js Modules

Working with Files and Directories

The file system is a core component of most back-end applications, enabling us to store, organize, and retrieve data. Node.js provides the `fs` module, a built-in library that allows us to interact with the file system programmatically. It supports tasks like reading and writing files, managing directories, and retrieving file details. The module offers both synchronous and asynchronous methods, including a Promise-based API for more readable and manageable code.

Promises: A quick recap

Recall that promises provide a structured way to handle asynchronous tasks in JavaScript. They enable chaining operations, managing errors effectively, and writing more maintainable code compared to callbacks. Using `.then()` for results and `.catch()` for errors, promises simplify the flow of complex asynchronous operations. With the `fs` module, we can handle a wide range of file-related tasks, such as:

- Managing logs and configurations.
- Creating and removing files or directories.
- Automating backups or processing user-generated files.

By mastering these features, we can create applications that efficiently handle file system operations while maintaining reliability and organization.

Reading and writing files

We've previously explored how to read and write files using the `fs` module, but let's take a closer look at these fundamental operations. Node.js provides both synchronous and asynchronous methods for these operations, allowing flexibility based on the needs of the application.

Asynchronous operations with Promises

The asynchronous methods in the `fs` module are often preferred for non-blocking I/O. Here's an example of reading the contents of a file, modifying it, and writing it to a new file asynchronously:

Files-Folder

example.txt

Hello, this is a sample text file.

It contains multiple lines of text for demonstration purposes. You can use it to test reading and writing operations in Node.js.

Index.js

```
const fs = require('fs').promises;
async function readAndWriteFile() {
  try {
    const data = await fs.readFile('example.txt', 'utf8');
    console.log("File content:", data);
    await fs.writeFile('output.txt', `Modified: ${data}`);
    console.log("Content written to output.txt");
  } catch (err) {
    console.error("Error:", err.message);
  }
}
readAndWriteFile();

console.log("This line runs immediately while file operations happen in the background, demonstrating asynchronous (non-blocking) behavior.");
```

Output:

C:\Program Files\nodejs\node.exe .\index.js

This line runs immediately while file operations happen in the background, demonstrating asynchronous (non-blocking) behavior.

File content: Hello, this is a sample text file.

It contains multiple lines of text for demonstration purposes.

You can use it to test reading and writing operations in Node.js.

Content written to output.txt

Explanation

- **Line 1:** Imports the `fs` module with `Promise` support.
- **Line 3:** Declares an asynchronous function to encapsulate file operations.
- **Line 5:** Reads the content of `example.txt` using `fs.readFile`. The second parameter specifies the encoding (`'utf8'`) to return the content as a string.
- **Line 8:** Writes the modified content to `output.txt` using `fs.writeFile`. The first parameter specifies the file name, and the second parameter specifies the content to write.
- **Lines 10–12:** Handles errors during reading or writing using a `try...catch` block.
- **Lines 15–16:** Calls the `readAndWriteFile` function. The log statement runs immediately without waiting for the file operations to finish, demonstrating asynchronous (non-blocking) behavior.

Synchronous operations

Synchronous methods are useful in scenarios where blocking the execution is acceptable, such as in scripts or tasks where operations must occur sequentially. Here's the synchronous version of the same example:

```
const fs = require('fs');
function readAndWriteFileSync() {
  try {
    const data = fs.readFileSync('example.txt', 'utf8');
    console.log('File content:', data);

    fs.writeFileSync('output.txt', `Modified: ${data}`);
    console.log('Content written to output.txt');
  } catch (err) {
    console.error('Error:', err.message);
  }
}
readAndWriteFileSync();
console.log("This line runs immediately after calling readAndWriteFileSync(), demonstrating synchronous behavior.");
```

Output

C:\Program Files\nodejs\node.exe .\index.js

File content: Hello, this is a sample text file.

It contains multiple lines of text for demonstration purposes.

You can use it to test reading and writing operations in Node.js.

Content written to output.txt

This line runs immediately after calling `readAndWriteFileSync()`, demonstrating synchronous behavior.

Explanation

- **Line 1:** Imports the `fs` module for file operations.
- **Line 3:** Declares a function for synchronous file operations.
- **Line 5:** Reads the content of `example.txt` synchronously using `fs.readFileSync`.
- **Line 6:** Logs the content of the file to the console.
- **Line 8:** Modifies the content and writes it to `output.txt` using `fs.writeFileSync`.
- **Line 9:** Logs a message confirming the successful write operation.
- **Line 10:** Handles any errors during file reading or writing with a `try...catch` block.
- **Lines 15–16:** Calls the `readAndWriteFileSync` function. The log statement is blocked and waits until the function completely finishes executing, demonstrating synchronous behavior.

Managing directories

Beyond reading and writing files, many applications require operations at the directory level, such as listing files in a folder, creating directories for organization, or cleaning up temporary directories. The `fs` module provides versatile methods to handle these tasks programmatically, enabling automation and efficient file system management.

Let's explore how we can list files in a directory and manage directories dynamically.

Listing files in a directory

The `fs.readdir` method is used to retrieve the contents of a directory. You can combine it with `fs.stat` to gather additional information about each file, such as its size and permissions. Here's an example:

Index.js

```
const fs = require('fs').promises;

async function listFiles(directory) {
  try {
    const files = await fs.readdir(directory);
    console.log(`Files in ${directory}:`);

    for (const file of files) {
      const stats = await fs.stat(`${directory}/${file}`);
      console.log(`${file} - Size: ${stats.size} bytes`);
    }
  } catch (err) {
    console.error("Error reading directory:", err.message);
  }
}

listFiles('./example-directory');
```

output:

Terminal

node index.js

Files in ./example-directory:

.DS_Store - Size: 6148 bytes

._DS_Store - Size: 120 bytes

._file1.txt - Size: 220 bytes

file1.txt - Size: 39 bytes

file2.txt - Size: 40 bytes

file3.txt - Size: 39 bytes

Explanation

- **Line 1:** Imports the `fs` module.
- **Line 3:** Declares an asynchronous function to handle directory operations.
- **Line 5:** Uses `fs.readdir` to retrieve file names in the specified directory.
- **Lines 8–11:** Iterates through each file, uses `fs.stat` to retrieve metadata (like size), and logs the file name and size.
- **Lines 12–14:** Handles errors such as missing or inaccessible directories.

Creating and removing directories

The `fs.mkdir` and `fs.rmdir` methods allow us to create and delete directories programmatically. Here's how to use them:

index.js

```
const fs = require('fs').promises;
async function manageDirectories() {
  try {
    await fs.mkdir('new-folder');
    console.log("Directory created: new-folder");

    await fs.rmdir('new-folder');
    console.log("Directory removed: new-folder");
  } catch (err) {
    console.error("Error managing directories:", err.message);
  }
}
manageDirectories();
```

output:

C:\Program Files\nodejs\node.exe .\index.js

Directory created: new-folder

Directory removed: new-folder

Explanation

- **Line 1:** Imports the `fs` module.
- **Line 3:** Declares the `manageDirectories` function to encapsulate directory operations.
- **Line 5:** Creates a directory named `new-folder` using `fs.mkdir`.
- **Line 8:** Removes the created directory using `fs.rmdir`.
- **Lines 10–12:** Catches and logs errors, such as attempting to remove a non-empty directory.

Working with Path, OS, and Util Modules

In back-end development, we often encounter tasks like managing file paths, adapting to system environments, and debugging code efficiently. Node.js provides core modules that make these tasks simpler and more reliable, saving time and effort in development. Some of the modules are:

- **path:** Helps manage file and directory paths consistently across operating systems, solving common challenges when working with file systems.
- **os:** Provides information about the system environment, such as memory usage, CPU details, and platform type, enabling applications to optimize performance or adapt based on available resources.
- **util:** Offers tools to streamline code, especially for handling asynchronous workflows and debugging complex applications.

These modules allow us to handle everyday development challenges efficiently and form an integral part of building scalable and maintainable Node.js applications.

The `path` module

The `path` module helps manage file paths consistently across different operating systems, ensuring reliability in both platform-specific and cross-platform applications. It provides methods to join, resolve, and parse paths, simplifying file and directory management. Here are some commonly used methods of the `path` module:

Method	Description	Example
<code>path.join</code>	Combines multiple path segments into one, ensuring proper separators for the platform.	<code>path.join('src', 'components', 'file.js')</code> → <code>'src/components/file.js'</code>
<code>path.resolve</code>	Resolves a sequence of paths into an absolute path.	<code>path.resolve('./src/file.js')</code> → <code>'/home/user/project/src/file.js'</code>
<code>path.basename</code>	Returns the last portion of a path (file name).	<code>path.basename('/src/file.js')</code> → <code>'file.js'</code>
<code>path.extname</code>	Retrieves the extension of a file from its path.	<code>path.extname('/src/file.js')</code> → <code>'.js'</code>
<code>path.dirname</code>	Returns the directory portion of a path.	<code>path.dirname('/src/file.js')</code> → <code>'/src'</code>

The following example demonstrates how to use the `path` module to construct, analyze, and manipulate file paths dynamically. Each method is applied to a practical use case to show its functionality.

Index.js

```
const path = require('path');

// Combine directory and file names
const combinedPath = path.join('folder', 'subfolder', 'file.txt');
console.log('Combined Path:', combinedPath);

// Resolve to an absolute path
const absolutePath = path.resolve('folder', 'subfolder', 'file.txt');
console.log('Absolute Path:', absolutePath);

// Extract the file name
const fileName = path.basename(combinedPath);
console.log('File Name:', fileName);

// Get the file extension
const fileExtension = path.extname(combinedPath);
console.log('File Extension:', fileExtension);

// Get the directory name
const directoryName = path.dirname(absolutePath);
console.log('Directory Name:', directoryName);
```

```
C:\Program Files\nodejs\node.exe .\index.js
```

```
Combined Path: folder\subfolder\file.txt
```

```
Absolute Path: C:\Users\HP\Desktop\WT-2026-27\folder\subfolder\file.txt
```

```
File Name: file.txt
```

```
File Extension: .txt
```

```
Directory Name: C:\Users\HP\Desktop\WT-2026-27\folder\subfolder
```

Explanation

- **Line 1:** Imports the `path` module, which provides utilities for working with file and directory paths.
- **Lines 4–5:** Uses `path.join` to combine path segments ('folder', 'subfolder', and 'file.txt') into a single path. This ensures the correct path separator (/ or \) for the operating system.
- **Lines 8–9:** Uses `path.resolve` to convert the relative path ('folder/subfolder/file.txt') into an absolute path. If the first argument is not an absolute path, it prepends the current working directory.
- **Lines 12–13:** Uses `path.basename` to extract the file name ('file.txt') from the combined path.
- **Lines 16–17:** Uses `path.extname` to retrieve the file extension ('.txt') from the combined path.
- **Lines 20–21:** Uses `path.dirname` to extract the directory portion of the absolute path. It returns everything up to the last separator.

The `os` module

The `os` module provides tools to retrieve system-level information, allowing applications to adapt to their environment or optimize performance. It is particularly useful for tasks like monitoring resources, tailoring application behavior, and gathering insights about the operating system. Here are some commonly used methods of the `os` module:

Method	Description	Example
<code>os.platform</code>	Returns the platform type, such as <code>'linux'</code> , <code>'win32'</code> , or <code>'darwin'</code> .	<code>os.platform()</code> → <code>'linux'</code>
<code>os.type</code>	Returns the name of the operating system, such as <code>'Linux'</code> or <code>'Windows_NT'</code> .	<code>os.type()</code> → <code>'Linux'</code>
<code>os.totalmem</code>	Returns the total memory of the system in bytes.	<code>os.totalmem()</code> → <code>17179869184</code> (16 GB)
<code>os.freemem</code>	Returns the available free memory in bytes.	<code>os.freemem()</code> → <code>8589934592</code> (8 GB)
<code>os.cpus</code>	Returns an array of objects containing details about each logical CPU core.	<code>os.cpus()</code> → <code>[{ model: 'Intel...', speed: 3600 }, ...]</code>

The following example demonstrates how to use the `os` module to gather and display system information dynamically, showing how it can assist in system monitoring and application configuration.

Index.js

```
const os = require('os');

// Log the platform and type
console.log('Platform:', os.platform());
console.log('OS Type:', os.type());

// Log total and free memory in MB
const totalMemory = (os.totalmem() / (1024 * 1024)).toFixed(2);
const freeMemory = (os.freemem() / (1024 * 1024)).toFixed(2);
console.log(`Total Memory: ${totalMemory} MB`);
console.log(`Free Memory: ${freeMemory} MB`);

// Log CPU information
console.log('CPU Information:', os.cpus());
```

Output:

C:\Program Files\nodejs\node.exe .\index.js

Platform: win32

OS Type: Windows_NT

Total Memory: 8139.86 MB

Free Memory: 1783.83 MB

CPU Information: (4) [{...}, {...}, {...}, {...}]

Explanation

- **Line 1:** Imports the `os` module for accessing system information.
- **Lines 4–5:** Logs the operating system platform (`os.platform`) and type (`os.type`).
- **Lines 8–9:** Calculates total and free memory in MB using `os.totalmem` and `os.freemem`. Divides by $(1024 * 1024)$ to convert bytes to MB and uses `toFixed(2)` for readability.
- **Line 14:** Logs CPU information, including details like model and speed, using `os.cpus()`.

The `util` module

The `util` module provides tools for debugging, working with asynchronous code, and handling objects more effectively. While `util.promisify` is one of its most notable functions, the module also includes other utilities that are useful in back-end development. Here are some commonly used methods of the `util` module:

Function	Description	Example
<code>util.promisify</code>	Converts callback-based functions into Promise-based ones.	<pre>const readFileAsync = util.promisify(fs.readFile);</pre>
<code>util.callbackify</code>	Converts Promise-based functions back into callback-based ones.	<pre>const cbFunction = util.callbackify(asyncFunction);</pre>
<code>util.inspect</code>	Formats and inspects objects for debugging.	<pre>console.log(util.inspect(obj, { depth: null, colors: true }));</pre>
<code>util.types</code>	Provides utilities for checking data types, like <code>isPromise</code> or <code>isRegExp</code> .	<pre>util.types.isPromise(myVar) → true</pre>
<code>util.format</code>	Formats strings in a printf-like manner.	<pre>console.log(util.format('Name: %s, Age: %d', 'Alice', 30));</pre>

The `promisify` and `callbackify` functions

The following example demonstrates how to use both `promisify` and `callbackify` to convert a file-reading function back and forth between callback-based and Promise-based workflows.

Index.js

```
const util = require('util');
const fs = require('fs');

// Step 1: Original callback-based function
// This function uses Node.js's `fs.readFile` to read a file and expects a callback.
function readFileCallback(filePath, callback) {
  fs.readFile(filePath, 'utf8', callback);
}
```

```

// Step 2: Promisify the callback-based function
// `util.promisify` converts the callback-based function into one that returns a Promise.
// The new function can now use modern Promise workflows like `then` and `catch`.
const readFilePromise = util.promisify(readFileCallback);

// Step 3: Using the promisified function
// We can now call the promisified function and handle the result using Promises.
readFilePromise('example.txt')
  .then((data) => console.log('File Content (Promise):', data)) // Logs the file content if successful.
  .catch((err) => console.error('Error (Promise):', err.message)); // Logs an error message if something
  goes wrong.

// Step 4: Converting the Promise-based function back to a callback-based one
// `util.callbackify` converts a Promise-based function into a callback-based version.
// This is useful for interacting with APIs or libraries that expect a callback-style function.
const readFileWithCallback = util.callbackify(readFilePromise);

// Step 5: Using the callback-based version
// We can now call the function in the traditional callback style.
readFileWithCallback('example.txt', (err, data) => {
  if (err) {
    console.error('Error (Callback):', err.message); // Handles any errors.
  } else {
    console.log('File Content (Callback):', data); // Logs the file content if successful.
  }
});

```

Output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

File Content (Promise): Hello, this is a sample text file.

It contains multiple lines of text for demonstration purposes.

You can use it to test reading and writing operations in Node.js.

File Content (Callback): Hello, this is a sample text file.

It contains multiple lines of text for demonstration purposes.

You can use it to test reading and writing operations in Node.js.

Explanation

- **Lines 1–2:** Imports the `util` and `fs` modules.
- **Lines 6–8:** Defines the original callback-based function, `readFileCallback`, which reads a file's content using the `fs.readFile` method.
- **Line 13:** Uses `util.promisify` to convert the callback-based function into `readFilePromise`, a Promise-based version.
- **Lines 17–19:** Demonstrates calling `readFilePromise` directly with `.then` and `.catch`, logging the file content or error.
- **Line 24:** Uses `util.callbackify` to convert `readFilePromise` back into a callback-based function, `readFileWithCallback`.
- **Lines 28–34:** Demonstrates how to use the callback-based function, logging the file content or handling any errors.

The `util.inspect` function

The `util.inspect` function is a valuable debugging tool in Node.js. It allows us to print and explore complex objects in a more readable format than `console.log`. By customizing options such as depth, colors, and visibility of hidden properties, `util.inspect` helps us gain deeper insights into our data structures during development.

Index.js

```
const util = require('util');

const obj = {
  name: 'Node.js',
  details: {
    type: 'Runtime',
    purpose: 'Server-side JavaScript',
    nested: { example: 'Deep structure' },
  },
  metadata: new Map([['version', 'v18.0.0'], ['author', 'Node.js Foundation']]),
};
// Log object with default options
console.log(util.inspect(obj));
// Log object with full depth, colors, and hidden properties
console.log(util.inspect(obj, {
  depth: null,
  colors: true,
  showHidden: true,
  compact: false
}));
```

Output:

C:\Program Files\nodejs\node.exe .\index.js

```

{
  name: 'Node.js',
  details: {
    type: 'Runtime',
    purpose: 'Server-side JavaScript',
    nested: { example: 'Deep structure' }
  },
  metadata: Map(2) { 'version' => 'v18.0.0', 'author' => 'Node.js Foundation' }
}

{
  name: 'Node.js',
  details: {
    type: 'Runtime',
    purpose: 'Server-side JavaScript',
    nested: {
      example: 'Deep structure'
    }
  },
  metadata: Map(2) {
    'version' => 'v18.0.0',
    'author' => 'Node.js Foundation'
  }
}

```

Explanation

- **Line 1:** Imports the `util` module to access `util.inspect`.
- **Lines 3–11:** Defines a complex object `obj` with nested properties and a `Map` for metadata.
- **Line 14:** Calls `util.inspect(obj)` with default options:
 - Outputs a concise, readable string representation of `obj`.
 - Limits the depth of nested objects for brevity.
- **Lines 17–22:** Calls `util.inspect` with custom options for detailed debugging:
 - `depth: null`: Displays all nested levels of the object.
 - `colors: true`: Adds color formatting to the output (if the terminal supports it).
 - `showHidden: true`: Includes non-enumerable properties in the inspection.
 - `compact: false`: Formats the output over multiple lines for better readability.

Introduction to Buffers

Traditional JavaScript focuses on handling text and arrays, which are not optimized for raw binary data. This limitation makes tasks such as reading binary files, processing image data, or handling network streams inefficient. Buffers were introduced to fill this gap, enabling Node.js to efficiently manage and process binary data.

What are buffers?

Buffers are fixed-size memory allocations used to store raw binary data in Node.js. Unlike regular JavaScript arrays or strings, buffers allow direct manipulation of binary data. They act as a bridge between JavaScript's high-level abstractions, such as strings, file system operations, streams, and networking, and the lower-level operations required for tasks like file I/O, streaming binary data, or handling network packets.

They are allocated in memory outside the V8 JavaScript engine's heap. This design choice allows Node.js to manage large amounts of data without being constrained by the engine's memory limits. Additionally, buffers offer direct control over memory, which is crucial for high-performance applications like streaming servers or processing large datasets.

They are especially useful when working with:

- File processing (e.g., images, videos, binary files).
- Networking protocols (e.g., encoding/decoding packets).
- Real-time data streams.

Creating buffers

We can create buffers in Node.js using three primary methods:

- **Buffer.alloc(size)**: Allocates a buffer of the specified size and initializes its memory to zero. This method is safer because it ensures that the memory is clean, but it may be slightly slower due to the initialization step.
- **Buffer.allocUnsafe(size)**: Allocates a buffer of the specified size without initializing its memory. This method is faster but can contain old data (garbage values) until it is explicitly overwritten, making it less secure for handling sensitive data.
- **Buffer.from(data)**: Creates a buffer from existing data, such as a string, array, or another buffer. The content of the input is encoded or copied directly into the new buffer.

These methods provide flexibility depending on the requirements of the application, whether you prioritize safety or performance. For example, use **Buffer.alloc** for handling sensitive information like passwords, and use **Buffer.allocUnsafe** for non-critical, high-performance operations, such as quickly processing large temporary data. Let's see how these methods work in practice with some examples.

Index.js

```
// Allocate a safe buffer (initialized to zero)
const safeBuffer = Buffer.alloc(10);

// Allocate an unsafe buffer (not initialized, may contain garbage data)
const unsafeBuffer = Buffer.allocUnsafe(10);

// Create a buffer from a string
const stringBuffer = Buffer.from('Hello, Buffers!');

// Log the created buffers
console.log('Safe Buffer:', safeBuffer);
console.log('Unsafe Buffer:', unsafeBuffer);
console.log('String Buffer:', stringBuffer);
```

output :

C:\Program Files\nodejs\node.exe .\index.js

Safe Buffer: Buffer(10) [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, buffer: ArrayBuffer(10), byteLength: 10, byteOffset: 0, length: 10, Symbol(Symbol.toStringTag): 'Uint8Array']

Unsafe Buffer: Buffer(10) [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, buffer: ArrayBuffer(8192), byteLength: 10, byteOffset: 256, length: 10, Symbol(Symbol.toStringTag): 'Uint8Array']

String Buffer: Buffer(15) [72, 101, 108, 108, 111, 44, 32, 66, 117, 102, 102, 101, 114, 115, 33, buffer: ArrayBuffer(8192), byteLength: 15, byteOffset: 272, length: 15, Symbol(Symbol.toStringTag): 'Uint8Array']

Explanation

- **Line 2:** Allocates a 10-byte buffer initialized with zeros, ensuring safe content.
- **Line 5:** Allocates a 10-byte buffer without initializing the memory, which may contain random (garbage) data. This approach is faster but riskier.
- **Line 8:** Creates a buffer from the string "Hello, Buffers!", encoding the data in UTF-8.
- **Lines 11–13:** Logs the content of each buffer for visualization.

Working with buffers

Buffers are highly versatile. Once created, we can manipulate them in a variety of ways to suit our needs. Let's look at some of the key operations we can perform on buffers, starting with writing data.

Writing to buffers

The `.write()` method allows controlled writing to a buffer. We can specify the data, position, and length.

Index.js

```
// Allocate a 60-byte buffer
const buffer = Buffer.alloc(60);

// Write a user's name at the start of the buffer
buffer.write('Name: John Doe');
```

```
// Write the user's email starting at position 20
buffer.write('Email: john@example.com', 20);
// Write only part of the user's account status starting at position 50
buffer.write('Status: Active', 50, 7); // Writes only 'Status:'
// Log the entire buffer content
console.log('Buffer Contents:', buffer.toString('utf8'));
```

Output:

C:\Program Files\nodejs\node.exe .\index.js

```
Buffer Contents: Name: John DoeEmail: john@example.comStatus:
```

Explanation

- **Line 2:** Allocates a 60-byte buffer using `Buffer.alloc(60)`. This provides sufficient space to store structured information such as name, email, and account status, with padding between sections.
- **Line 5:** Writes `'Name: John Doe'` at the beginning of the buffer:
 - The data occupies the first 14 bytes (0–13).
 - No offset or length is specified, so the string starts at position 0 and writes until the string ends.
- **Line 8:** Writes `'Email: john@example.com'` starting at position 20:
 - This ensures the email is stored separately from the name, occupying bytes 20–43.
 - The offset parameter (20) specifies where the writing begins in the buffer.
- **Line 11:** Writes only `'Status:'` (the first 7 characters of `'Status: Active'`) starting at position 50.
- **Line 14:** Logs the entire buffer's content as a UTF-8 string.

Reading from buffers

The `.toString()` method is the most common way to convert a buffer's binary data into a readable format. We can specify an encoding (e.g., `'utf8'`, `'base64'`, `'hex'`) and optionally include a start and end index to read a specific portion of the buffer.

Index.js

```
const buffer = Buffer.from('Learning Buffers in Node.js');
// Read the entire buffer as a UTF-8 string
console.log('Full String:', buffer.toString('utf8'));
// Read a portion of the buffer (first 8 characters)
console.log('Partial String:', buffer.toString('utf8', 0, 8));
// Read the buffer content from a specific index till the end
console.log('From Index 9:', buffer.toString('utf8', 9));
// Read the buffer in hexadecimal format
console.log('Hexadecimal Format:', buffer.toString('hex'));
// Read the buffer in base64 encoding
console.log('Base64 Encoding:', buffer.toString('base64'));
```

output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

Full String: Learning Buffers in Node.js

Partial String: Learning

From Index 9: Buffers in Node.js

Hexadecimal Format: 4c65617226e696e67204275666665727320696e204e6f64652e6a73

Base64 Encoding: TGVVhcm5pbmcgQnVmZmVycyBpbjBOb2RlLmpz

Explanation

- **Line 1:** A buffer is created using `Buffer.from` to store the string as binary data, allowing for flexible manipulation and encoding.
- **Line 4:** The entire buffer content is read and converted into a UTF-8 string using `toString`. This retrieves all the data as a human-readable string.
- **Line 7:** A specific portion of the buffer is read by specifying start and end indices in `toString`.
- **Line 10:** The buffer content is read from a given start index to the end of the buffer using `toString` with a single index parameter.
- **Line 13:** The buffer is converted into a hexadecimal string, where each byte is represented in base 16. This encoding is valuable for debugging or interpreting binary data in a readable format.
- **Line 16:** The buffer is encoded into a Base64 string, a format commonly used for transmitting binary data over text-based systems.

Accessing and iterating over buffers

Buffers behave like arrays, allowing us to access individual bytes directly or iterate over their contents using loops. For example, this flexibility enables tasks such as reading specific binary values or processing a file's data byte by byte.

Index.js

```
const buffer = Buffer.from('Hello');  
// Access individual bytes  
console.log('First byte:', buffer[0]);  
// Iterate over all bytes  
for (const byte of buffer) {  
  console.log('Byte:', byte);  
}
```

Output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

First byte: 72

Byte: 72

Byte: 101

Byte: 108

Byte: 111

Explanation

- **Line 1:** Creates a buffer with raw byte values corresponding to the string 'Hello'.
- **Line 4:** Accesses the first byte using array-like indexing (`buffer[0]`), which returns the numeric value of the byte (72 for 'H' in the default UTF-8 encoding).
- **Lines 7–9:** Loops through the buffer with a `for...of` loop, logging each byte's numeric value.

Copying buffers

In many applications, we need to duplicate or reuse data. The `Buffer.copy` method allows us to efficiently copy data from one buffer to another without affecting the original.

Index.html

```
// Create two buffers
```

```
const sourceBuffer = Buffer.from('Source Buffer Data');
```

```
const targetBuffer = Buffer.alloc(20);
```

```
// Copy sourceBuffer content into targetBuffer
```

```
sourceBuffer.copy(targetBuffer, 0, 0, sourceBuffer.length);
```

```
console.log('Source Buffer:', sourceBuffer.toString());
```

```
console.log('Target Buffer:', targetBuffer.toString());
```

output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

```
Source Buffer: Source Buffer Data
```

```
Target Buffer: Source Buffer Data
```

Explanation

- **Line 2:** Creates the `sourceBuffer` with the string data to be copied.
- **Line 3:** Allocates the `targetBuffer`, a blank buffer with a fixed size.
- **Line 6:** Copies data from `sourceBuffer` to `targetBuffer` using the `Buffer.copy` method, specifying source and target offsets and length.
- **Line 8:** Logs the contents of `sourceBuffer` to verify the original data.
- **Line 9:** Logs the contents of `targetBuffer` to verify the copied data.

Combining buffers

Sometimes, we need to merge multiple buffers into one, such as when aggregating data from different sources. The `Buffer.concat` method provides a simple way to achieve this.

Index.js

```
// Create two buffers containing text
const buffer1 = Buffer.from('Hello, ');
const buffer2 = Buffer.from('World!');

// Combine the two Buffers
const combinedBuffer = Buffer.concat([buffer1, buffer2]);

// Log the combined buffer as a string
console.log('Combined Buffer:', combinedBuffer.toString('utf8'));
```

Output:

```
C:\Program Files\nodejs\node.exe .\index.js
Combined Buffer: Hello, World!
```

Explanation

- **Lines 2–3:** Creates two buffers, each containing part of a sentence.
- **Line 6:** Combines the buffers into a new one using `Buffer.concat`.
- **Line 9:** Converts the resulting buffer into a UTF-8 string for logging.

Comparing buffers

When working with binary data, it's often necessary to compare buffers. The `.equals()` method makes it easy to check if two buffers contain identical data.

Index.js

```
// Create three buffers
const buffer1 = Buffer.from('Node.js');
const buffer2 = Buffer.from('Node.js');
const buffer3 = Buffer.from('Buffers');

// Compare buffers for equality
console.log('Are Buffers Equal:', buffer1.equals(buffer2)); // true

// Compare buffers with different content
console.log('Are Buffers Equal:', buffer1.equals(buffer3)); // false
```

output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

Are Buffers Equal: true

Are Buffers Equal: false

Explanation

- **Lines 2–4:** Initializes three buffers with different contents.
- **Line 7:** Compares `buffer1` and `buffer2`, which have identical content, returning `true`.
- **Line 10:** Compares `buffer1` and `buffer3`, which differ, returning `false`.

Slicing buffers

The `Buffer.slice` method enables us to work with a specific portion of the data without duplicating memory. This is particularly useful for streaming or processing large datasets incrementally.

Index.js

```
const buffer = Buffer.from('Hello, Buffers!');  
// Create a sub-buffer  
const subBuffer = buffer.slice(7, 14);  
console.log('Sub-buffer:', subBuffer.toString('utf8'));
```

output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

Sub-buffer: Buffers

Explanation

- **Line 1:** Creates a buffer from the string "Hello, Buffers!" using `Buffer.from`.
- **Line 4:** Slices the buffer to create a sub-Buffer from index `7` to `14`.
- **Line 6:** Converts the sub-buffer to a string using `.toString('utf8')`.

Working with Streams

Managing data efficiently is a key aspect of many modern applications, especially when dealing with large files, real-time data, or network requests. Node.js streams provide a way to handle data piece by piece, making it possible to process large amounts of information without overloading memory.

Streams are commonly used in scenarios like:

- **File handling:** Reading and writing large files efficiently.
- **Web applications:** Managing incoming requests and sending responses incrementally.
- **Real-time applications:** Enabling continuous data flow for features like live chat or data feeds.

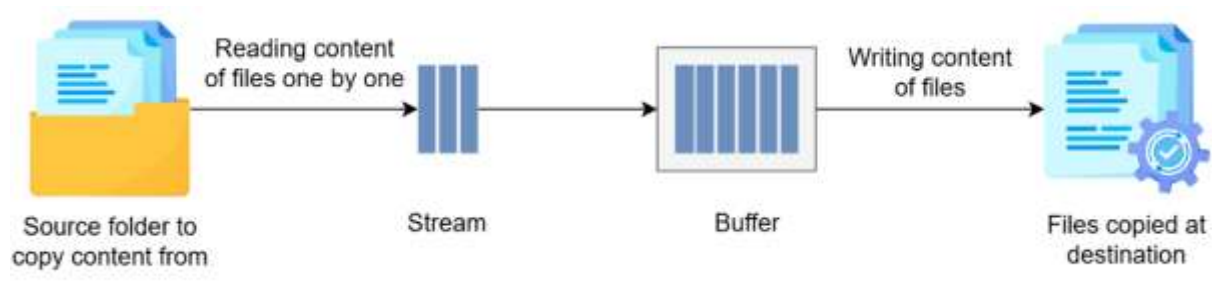
Unlike traditional methods that require loading all data at once, streams allow us to process data as it arrives. This approach fits well with Node.js's event-driven model and helps build fast, scalable applications.

What are streams?

Streams in Node.js are abstract interfaces for working with data that is read or written sequentially. They represent a continuous flow of information, allowing data to be processed incrementally rather than all at once. This makes streams particularly useful when handling large datasets or real-time data where loading everything into memory would be impractical.

At their core, streams enable efficient, non-blocking I/O operations by breaking data into manageable chunks. This approach ensures that applications remain responsive, even when dealing with significant amounts of data.

To achieve this, streams rely on buffers to handle data in chunks. A buffer temporarily holds small pieces of data as it moves through the stream, enabling efficient, non-blocking processing. This design ensures that even large files or data streams can be processed without overwhelming the system's memory.



Working of streams

Node.js provides four main types of streams, each serving a specific purpose:

- **Readable streams:** Designed for reading data in chunks from a source, such as reading lines from a file or receiving data over a network connection.
- **Writable streams:** Allow data to be written incrementally to a destination, such as saving logs to a file or sending responses in an HTTP server.
- **Duplex streams:** Combine the functionality of readable and writable streams, enabling data to be read and written simultaneously. They are commonly used in network communication, such as sockets.
- **Transform streams:** A special type of duplex stream that reads data, processes or transforms it (e.g., compressing or encrypting), and then writes the transformed output.

The readable streams

Readable streams allow us to read data in chunks from a source, making them efficient for handling large datasets or continuous data flows. These streams can originate from various sources, such as:

- Files
- HTTP requests

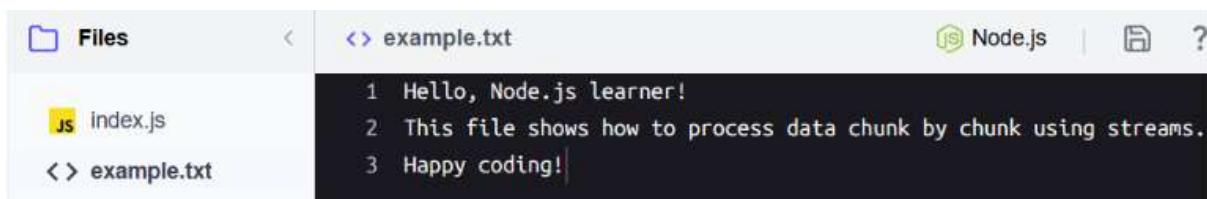
- Network sockets

When working with a readable stream, data is read in chunks, allowing large datasets to be processed incrementally. The size of each chunk is controlled by the `highWaterMark` option, which specifies the maximum number of bytes to read at a time. By default, this value is 16 KB for most streams, but it can be adjusted to suit specific use cases. Readable streams operate in an event-driven manner, emitting events like:

- `data`: Triggered when a chunk of data is available for processing.
- `error`: Triggered if an error occurs during reading.

The following example demonstrates reading data incrementally from a file using a readable stream created with `fs.createReadStream`. This method processes data in chunks as it appears in the file, making it especially useful for handling large files efficiently without loading the entire content into memory.

Example.txt:



```
Files < example.txt Node.js ?
js index.js
< > example.txt
1 Hello, Node.js learner!
2 This file shows how to process data chunk by chunk using streams.
3 Happy coding!
```

Index.js

```
const fs = require('fs');
// Create a readable stream
const readableStream = fs.createReadStream('example.txt', { encoding: 'utf8', highWaterMark: 16 *
1024 });
// Listen for 'data' events
readableStream.on('data', (chunk) => {
  console.log('Received chunk:', chunk);
});
// Handle errors
readableStream.on('error', (err) => {
  console.error('Error reading file:', err.message);
});
```

Output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

Received chunk: Hello, this is a sample text file.

It contains multiple lines of text for demonstration purposes.

You can use it to test reading and writing operations in Node.js.

Explanation

- **Line 1:** The `fs` module is imported to access file system operations.
- **Line 4:** The `fs.createReadStream` method creates a stream to read the `example.txt` file.
 - The `encoding` option is set to `utf8` to interpret the file content as a string.
 - The `highWaterMark` option sets the chunk size to 16 KB ($16 \times 1024 = 16,384$ bytes). This means that each `data` event will emit up to 16KB of content. This matches the default value used by most readable streams in Node.js.
- **Lines 7–9:** The `data` event listener is attached to the readable stream. Whenever a chunk of data becomes available, the callback function logs the chunk to the console. This event-driven mechanism ensures that the application only processes data as it arrives, avoiding unnecessary memory usage.
- **Lines 12–14:** The `error` event listener handles any issues during file reading, such as file access errors or missing files. By logging the error message, we can identify and debug issues without causing the application to crash.

The writable streams

Writable streams allow us to write data incrementally to a destination, making them efficient for handling large outputs or continuous data flows. These streams are widely used in various scenarios, such as:

- Writing HTTP responses in a web server.
- Sending data over network sockets.

One common example is `fs.createWriteStream`, which is used to write data to files incrementally. The following example demonstrates how to use `fs.createWriteStream` to write multiple chunks of data to a file:

Index.js

```
const fs = require('fs');
// Create a writable stream
const writableStream = fs.createWriteStream('output.txt');
// Write chunks of data
writableStream.write('Hello, ');
writableStream.write('Streams in Node.js!');
writableStream.end(); // Signal the end of writing
console.log('Data written to file successfully.');
```

Output:

```
C:\Program Files\nodejs\node.exe .\index.js
Data written to file successfully.
```

Explanation

- **Line 1:** Imports the `fs` module.
- **Line 4:** Creates a writable stream for the file `output.txt`.
- **Lines 7–8:** Writes multiple chunks of data to the writable stream using the `write` method.
- **Line 9:** Signals the end of writing with the `end` method, ensuring all data is flushed.

The duplex streams

Duplex streams are a unique type of stream in Node.js that allow simultaneous reading and writing of data. This makes them ideal for scenarios where data flows bi-directionally, such as network communication using sockets or building custom data processors.

Unlike separate readable and writable streams, duplex streams combine both functionalities into a single object. This means that data can be read and written independently, often through distinct events and methods.

Key scenarios where duplex streams are used are:

- **Network communication:** Duplex streams are commonly used with sockets (e.g., `net.Socket`) to enable bidirectional communication, such as in a chat application.
- **Custom data processing:** We can create duplex streams for scenarios requiring both input and output, like converting data formats or implementing communication protocols.

The transform streams

Transform streams are a special type of duplex stream that can read, modify, and write data simultaneously. They are often used for tasks like compressing files, encrypting data, or converting formats. In the following example, we use Node.js's `stream.Transform` class to create a Transform stream that converts incoming data to uppercase before writing it out.

Index.js

```
const { Transform } = require('stream');
// Create a Transform stream
const upperCaseTransform = new Transform({
  transform(chunk, encoding, callback) {
    // Convert the chunk to uppercase and push it to the writable side
    this.push(chunk.toString().toUpperCase());
    callback();
  }
});
// Use the transform stream
upperCaseTransform.write('hello');
upperCaseTransform.write('world');
upperCaseTransform.end();
// Read transformed data
upperCaseTransform.on('data', (chunk) => {
  console.log(chunk.toString());
});
```

Output:

```
C:\Program Files\nodejs\node.exe .\index.js
```

HELLO WORLD

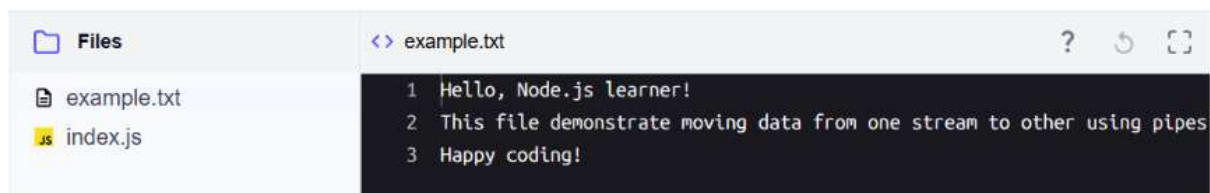
Explanation

- **Line 1:** Import the `Transform` class from Node.js's `stream` module.
- **Line 4:** Create a new Transform stream by instantiating the `Transform` class. The Transform stream is defined with a `transform` method passed in the options object.
 - **Lines 5–10:** Define the `transform` method:
 - **Parameters:**
 - **chunk:** A portion of data being processed. In this case, it's a chunk of text written to the stream.
 - **encoding:** The encoding type of the chunk (e.g., `utf8`). For binary streams, this parameter is ignored.
 - **callback:** A function that signals the completion of the transformation for the current chunk.
 - **Logic:**
 - `this.push(chunk.toString().toUpperCase());` Converts the chunk to uppercase and pushes the transformed data to the writable side of the stream.
 - `callback();` Indicates the chunk transformation is complete. Without calling this, the stream will not process additional chunks.
- **Lines 13–14:** Write data chunks ("hello" and "world") to the Transform stream using the `write` method.
- **Line 15:** End the stream using `end()`, signaling no more data will be written.
- **Line 18–20:** Listen for the `data` event on the stream to read transformed data as it flows through. Each transformed chunk is logged to the console.

This example demonstrates how the `stream.Transform` class provides a powerful way to manipulate data as it flows through streams. By defining the `transform` method, we can implement custom logic to modify or process data on the fly, making it especially useful for tasks like data formatting, compression, or encryption.

Combining streams with piping

Streams can be connected using the `.pipe()` method, allowing data to flow seamlessly from one stream to another. In the example below, we connect the readable stream to the writable stream using pipe.



```
Files
example.txt
index.js
example.txt
1 Hello, Node.js learner!
2 This file demonstrate moving data from one stream to other using pipes
3 Happy coding!
```

Index.js

```
const fs = require('fs');  
// Pipe data from a readable stream to a writable stream  
fs.createReadStream('example.txt')  
  .pipe(fs.createWriteStream('output.txt'));  
console.log('File copied successfully.');
```

output:

```
C:\Program Files\nodejs\node.exe .\index.js  
File copied successfully.
```

Explanation

- **Line 1:** Imports the `fs` module.
- **Lines 4–5:** Uses `.pipe()` to connect the readable stream for `example.txt` to the writable stream for `output.txt`.
- **Line 7:** Logs a success message once the copying process completes.

npm and Package Management

Introduction to npm

What is npm?

npm is an essential tool included with Node.js that simplifies how developers use and share code. It serves two main purposes:

1. **Package management:** npm allows us to easily install, update, and manage dependencies—libraries or tools that enhance the functionality of Node.js applications.
2. **Online registry:** It provides access to a massive repository of reusable JavaScript packages created and shared by developers worldwide.

When we install Node.js, npm is bundled with it, providing a command-line interface (CLI) to interact with the npm registry and manage project dependencies directly.

This makes npm invaluable for tasks such as:

- **Installing libraries:** Add functionality to our projects with commands like `npm install`.
- **Automating tasks:** Define scripts to streamline workflows, such as running tests or starting applications.
- **Sharing code:** Publish reusable packages to the npm registry, making them available to the global developer community or within teams.

In short, npm plays a central role in the Node.js ecosystem, helping developers manage dependencies and share reusable tools.

Getting started with npm

To harness the full power of npm in projects, we need a special file called `package.json`. Think of it as the blueprint for the Node.js project. This file stores essential information, such as:

- The list of dependencies installed via npm.
- Metadata like the project name, version, and description.
- Custom scripts to automate tasks like starting a server or running tests.

Every Node.js project that uses npm starts with a `package.json` file. Let's see how to create one.

1. Open a terminal and navigate to a new or existing project folder:

```
2. mkdir my-project && cd my-project
```

3. Run the following command to initialize the project:

```
4. npm init
```

5. Answer the prompts. Here's a typical list of questions we might see:
 - **Package name:** The project's name (defaults to the folder name).
 - **Version:** The initial version number (defaults to `1.0.0`).
 - **Description:** A short description of the project.
 - **Entry point:** The starting file for the app (e.g., `index.js`).
 - **Test command:** Command to run tests (optional; can be left blank).
 - **Git repository:** Link to the Git repository (optional).
 - **Keywords:** Words to describe the project (useful for discovery if published to npm).
 - **Author:** Your name or organization.
 - **License:** The license type (defaults to `ISC`).

And that's it! We have successfully generated a `package.json` file in our project directory.

```
1 {  
2   "name": "my-project",  
3   "version": "1.0.0",  
4   "description": "",  
5   "main": "index.js",  
6   "scripts": {  
7     "test": "echo \"Error: no test specified\" && exit 1"  
8   },  
9   "keywords": [],  
10  "author": "",  
11  "license": "ISC"  
12 }
```

A typical `package.json` file created with `npm init`, containing default fields and values

Let's explore the fields of the `package.json` file one by one:

- **name:** Represents the name of our project. By default, it is set to the folder name where the project resides, but we can also change it. This field is essential if we plan to publish the package on npm, as it must be unique across the npm registry.

- **version**: Indicates the version of our project, typically following semantic versioning (e.g., **1.0.0**). It allows us to track changes and updates to our project.
- **description**: An optional short description that briefly explains what our project does. It helps others understand our project when browsing npm or documentation.
- **main**: Specifies the file Node loads when the package is required from its root. It is the package entry point, not necessarily the app's runtime startup file, applications often use **index.js** or another file for that.
- **scripts**: Defines custom command-line scripts that we can run using **npm run <script-name>**. We'll explore it further shortly.
- **keywords**: An array of strings describing our project. These keywords improve discoverability on npm if we publish our project. Example: **"keywords": ["nodejs", "example", "npm"]**.
- **author**: Specifies the author or maintainer of the project. This field is particularly useful when working on open-source or collaborative projects.
- **license**: Indicates the type of license under which our project is distributed. By default, it is set to **"ISC"**, which is a permissive open-source license. We can choose another license as needed.

In future lessons, we'll see how **package.json** can track external libraries our project might need, making it even more powerful as our application grows.

Adding scripts

Scripts in **package.json** are an essential way to automate tasks and streamline workflows in a Node.js project. They let us define commands that can be executed with **npm run**. We can use scripts to automate tasks like testing, building, or starting our application. Here's an example of a script to start an app:

```
1 "scripts": {
2   "start-app": "node app.js"
3 }
```

This script can be executed with:

```
1 npm run start-app
```

npm recognizes certain special script names in the **package.json** file with predefined behaviors. These scripts allow for simplified commands or automatic execution during specific stages of the development process. Here are some of the most common script names:

- **start**: It can be run with **npm start** command (without **run**). If no **start** script is defined, npm defaults to running the **node server.js**.
- **test**: It can be run with **npm test** command if a **test** script is defined.
- **pre<name> and post<name> hooks**: npm allows **pre<name>** and **post<name>** hooks for any script, running before or after the specified script. For example:
 - **prestart**: It runs before executing the **start** script.
 - **posttest**: It runs after executing the **test** script.

For instance, here's how we can use special script names in a `package.json` file:

```
1 "scripts": {  
2   "start": "node app.js",  
3   "test": "echo 'Running tests...'",  
4   "prestart": "echo 'Preparing to start the app...'",  
5   "posttest": "echo 'Test complete!'"  
6 }
```

A `package.json` file demonstrating the use of special script names and hooks

npm includes additional special script names, such as `install`, `prepublishOnly`, `preversion`, and `postversion`. These can be used for setup, publishing, or versioning tasks.

Working with npm Packages

Installing packages

Before using an external library in our project, we must install it. npm makes this process seamless, whether installing locally for a specific project or globally for system-wide tools.

Installing a package locally

Local installations are tied to a specific project and are the most common way to use npm packages. They allow us to include libraries specific to our project's needs without affecting other projects. For example, we can install the `axios` library to simplify making HTTP requests in our application.

npm install axios

After running this command, the following happens:

1. npm adds the `axios` library to a `node_modules` directory in our project folder. This folder contains all locally installed packages and their dependencies.
2. By default, `npm install <package>` updates the `package.json` file, adding the package to the dependencies section. (Flags like `--save-dev` or `--no-save` change this behavior.):

```
3. "dependencies": {  
4.   "axios": "^1.7.8"  
5. }
```

The dependencies section in `package.json` is a critical part of npm's functionality. It serves as a record of all the external libraries the project requires to function. For each library, it includes:

- **The library name:** For example, `"axios"`.
- **The version range:** Specifies which versions of the library are compatible (e.g., `"^1.7.8"`).

When we share our project with others or move it to a different environment, the `dependencies` section ensures that anyone working with the project can install the exact libraries it needs. When we run `npm install`, npm reads the `dependencies` section and installs all listed libraries into the `node_modules` folder. This approach ensures:

- **Consistency:** The project behaves the same way across different systems.
- **Reproducibility:** Any team member or deployment process can recreate the environment without manual steps.
- **Simplified maintenance:** We don't need to remember or document which libraries are required—they're all tracked in `package.json`.

For example, if `package.json` includes:

```
1 { "dependencies": {
2   "chalk": "^5.0.0",
3   "axios": "^1.3.0"
4 }
```

Running `npm install` will download and install both `chalk` and `axios` along with their compatible dependencies.

Installing a package globally

While most libraries are installed locally to specific projects, some tools are better suited for global installation. Global packages are stored in a system-wide location and are available from any project or terminal session. It is ideal for tools that are not directly required by our project but improve our workflow and for CLI commands that we want to use across multiple projects. Examples of globally installed tools include:

- `nodemon`: Automatically restarts the Node.js app when files change during development.
- `eslint`: Enforces coding standards and helps maintain code quality.

To install a package globally, use:

```
npm install -g <package-name>
```

For example, to install `nodemon` globally use the command:

```
npm install -g nodemon
```

We can confirm the installation by checking the version of the package. For example:

```
nodemon --version
```

Global packages are installed in a location outside our project folder, typically in `/usr/local/lib/node_modules` on Linux/Mac systems or `%APPDATA%\npm\node_modules` on Windows.

Adding `devDependencies`

Some dependencies are only needed during development and are not required at runtime in the production environment. For example, build tools like `Webpack` and `Babel`, testing libraries like `Jest` or linters like `ESLint` are typically installed as `devDependencies`. Production installs commonly omit `devDependencies` (for example, using `npm install --omit=dev`). If a deployment installs all dependencies, `devDependencies` may still be present. To install such a package, use the `--save-dev` flag with the `npm install` command:

```
npm install <package-name> --save-dev
```

For example, to install the `eslint` package use:

```
npm install eslint --save-dev
```

Executing the command adds the `eslint` package to the `package.json` file, creating or modifying the `devDependencies` section. For example, the `package.json` file with `devDependencies` looks like below:

```
1 {  
2   "dependencies": {  
3     "axios": "^1.7.8"  
4   },  
5   "devDependencies": {  
6     "eslint": "^8.19.0"  
7   }  
8 }
```

Hands-on practice with npm packages

Let's demonstrate how to use `axios`, a library for making HTTP requests, as an example of working with npm packages. We'll install and integrate it into a Node.js project in the playground below.

Step 1: Add code to `index.js`

Start by adding the following code to the `index.js` file:

Using the `axios` package to make an HTTP request and handle the response.

```
const axios = require('axios');  
  
// Fetching data from a public API  
axios.get('https://api.agify.io?name=michael')  
  .then(response => {  
    // Logging the response data  
    console.log('Response Data:', response.data);  
  })  
  .catch(error => {  
    // Logging an error message if the request fails  
    console.error('Error fetching data:', error.message);  
  });
```

Explanation

- **Line 1:** Import the `axios` library into our project using `require`. Note that this works because `axios` provides CommonJS compatibility; import style can vary by package and version.
- **Line 4:** Use the `axios.get` method to send a GET request to the public API (<https://api.agify.io?name=michael>). This API predicts a person's age based on their name.

- **Lines 5–8:** The `then` block runs if the request is successful, and the response data is logged to the console.
- **Lines 9–12:** The `catch` block handles errors (e.g., if the API is unreachable), preventing an unhandled promise rejection for this request.

Step 2: Start the playground

Click the "Run" button to launch the environment. The playground is pre-configured with a Node.js project.

Step 3: Install `axios`

Run the following command in the terminal to add `axios` to the project:

```
npm install axios
```

This command:

- Downloads `axios` and its dependencies.
- Adds `axios` to the dependencies section of the `package.json` file.

Step 4: Run the script

Execute the `index.js` file to see the output:

```
node index.js
```

Expected output

You should see the following response logged in your terminal, confirming the API request was successful.

Response Data: { name: 'michael', age: <number>, count: <number> }

Managing Dependencies

Dependencies are an integral part of modern Node.js development. They enable developers to reuse code, avoid reinventing the wheel, and build applications faster. However, managing dependencies can become challenging as projects grow in complexity. To manage dependencies effectively, npm provides two essential resources:

- `node_modules`: A folder that stores the installed libraries and their dependencies.
- `package-lock.json`: A file that ensures consistency and stability in dependency versions across environments.

Understanding `node_modules`

The `node_modules` folder is automatically created in our project when we install dependencies. Let's explore its role in detail:

- **Stores dependencies:** When we install a library using `npm install`, npm fetches the package and stores it in the `node_modules`.

- **Handles nested dependencies:** Many packages rely on other libraries. npm resolves these nested dependencies and places them in `node_modules`, though npm's deduplication and hoisting may place some packages higher in the tree rather than duplicating them.
- **Ensures project functionality:** By keeping all required libraries locally, npm ensures that the project works consistently regardless of the environment.

Here are some key facts about `node_modules`:

- **Size considerations:** For projects with numerous dependencies, `node_modules` can become large.
- **Recreation:** If the `node_modules` folder is deleted, it can be recreated by executing `npm install` command.
- **Exclusion from version control:** Since it can be regenerated using `npm install`, this folder is usually excluded from version control by adding it to `.gitignore`.

Suppose we installed the `axios` package. The `node_modules` folder structure might look like this:

```
1 node_modules/
2 └─ axios/
3 └─ follow-redirects/
4 └─ ...
```

The `node_modules` folder includes the installed package (`axios`) and its dependencies (`follow-redirects`)

Understanding `package-lock.json`

The `package-lock.json` file works alongside `node_modules` to lock dependency versions and maintain project stability. Let's explore it in more detail:

- **Records exact versions:** While `package.json` lists version ranges (e.g., `^1.0.0`), `package-lock.json` records the exact versions of each installed dependency.
- **Tracks nested dependencies:** It lists not only direct dependencies but also their sub-dependencies.
- **Enables reproducible installs:** `npm ci` is the preferred command for exact, deterministic installs from `package-lock.json` in CI environments; `npm install` can, in some cases, still modify the lockfile.

Here are some key features:

- **Dependency integrity:** It records `checksums` to verify the integrity of downloaded libraries.
- **Stability across environments:** It ensures that the same versions are used in development, testing, and production.

Here's an example of `package-lock.json` for the `axios` package:

```

1 {
2   "name": "my-project",
3   "lockfileVersion": 2,
4   "dependencies": {
5     "axios": {
6       "version": "1.7.8",
7       "resolved": "https://registry.npmjs.org/axios/-/axios-1.7.8.tgz",
8       "integrity": "sha512-abc123..."
9     },
10    "follow-redirects": {
11      "version": "1.14.9",
12      "resolved": "https://registry.npmjs.org/follow-redirects/-/follow-redirects-1.14.9.tgz",
13      "integrity": "sha512-def456..."
14    }
15  }
16 }

```

The package-lock.json file includes exact version details for axios and its dependencies

These two components complement each other to ensure consistency. While the `node_modules` contains the actual code files for all installed packages, enabling the project to run, the `package-lock.json` acts as a detailed blueprint of the dependency tree, ensuring consistency across machines and environments.

Managing dependencies effectively

Managing dependencies is a crucial aspect of maintaining a stable and functional Node.js project. It ensures that the application has access to the necessary libraries while maintaining consistency across different environments. Let's see how to manage these dependencies using npm.

Adding new dependencies

To install a new dependency, run the following command in the terminal:

```
npm install <package-name>
```

By default, this installs the latest version of the package available in the npm registry. To install a specific version of a dependency, specify the version number:

```
npm install <package-name>@<version>
```

This command:

- Adds the package to the `node_modules` directory.
- Updates the `package.json` file to include the package name and version.
- Updates the `package-lock.json` file to lock the exact version and its sub-dependencies.

Using specific versions allows you to maintain compatibility and control over your project dependencies.

Checking installed packages with `npm list`

The `npm list` command shows the dependencies installed in the project and their versions. It provides a tree-like structure, displaying the relationships between packages.

Here are some common use cases for the `npm list` command:

1. List all dependencies in the project:

`npm list`

2. Check a specific package:

`npm list chalk`

Displays the installed version of `chalk`.

3. Show dependencies up to one level deep:

`npm list --depth=1`

1. Displays the top-level dependencies along with their immediate sub-dependencies, if any, using tree connectors such as `|` and `└─`).

Use `npm list` to verify installed versions, debug issues, or inspect global packages with the `-g` flag.

Updating dependencies

Dependencies evolve over time as libraries introduce new features, improvements, and fixes. npm provides commands to help us keep them up to date. To update a specific dependency, run the following command in the terminal:

`npm update <package-name>`

This command:

- Updates the package in `node_modules` to the latest version allowed by the range specified in `package.json`. This stays within the declared range and may also update transitive dependencies when their parent packages allow it.
- Modifies `package-lock.json` to reflect the updated version, if the resolved version changed. Leaves version ranges in `package.json` unchanged.
- Leaves the version range in `package.json` unchanged.

Checking for outdated dependencies

To check for outdated dependencies, run the following command in the terminal:

`npm outdated`

This command generates a table showing:

- **Current:** The currently installed version of the package.
- **Wanted:** The latest version that satisfies the range specified in `package.json`.
- **Latest:** The most recent version of the package available on npm.

For example, the output might look like this:

1	Package	Current	Wanted	Latest	Location
2	axios	1.7.8	1.8.0	2.0.0	my-project

The `npm outdated` command shows which packages can be updated

Removing dependencies

To uninstall a dependency, use the following command:

`npm uninstall <package-name>`

This command:

- Removes the package from `node_modules`.
- Updates `package.json` to remove the dependency entry.
- Updates `package-lock.json` to reflect the removal.
- npm re-resolves the dependency tree and removes any now-unneeded sub-dependencies from `node_modules` as part of that updated graph.

Reinstalling dependencies

If `node_modules` is accidentally deleted or corrupted, we can regenerate it by running:

`npm install`

Exploring Commonly Used npm Packages

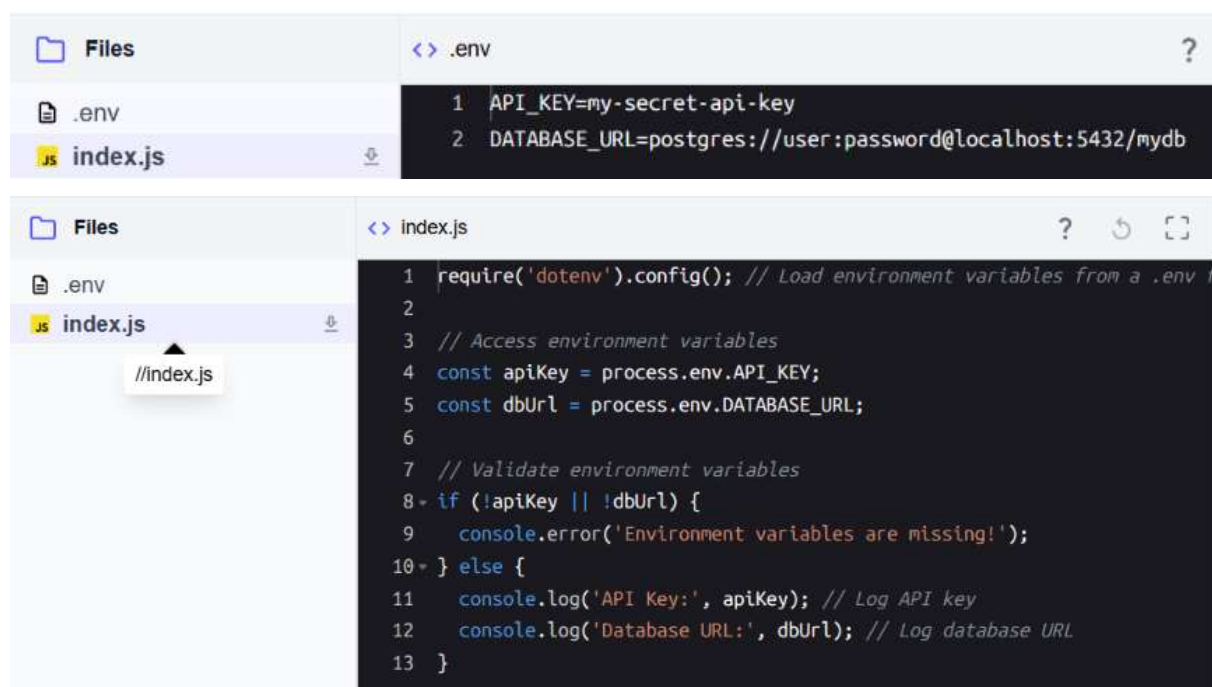
Node.js development is greatly enhanced by the vast array of npm packages available. Some packages are so essential that they're considered must-know tools for any developer. In this lesson, we'll survey five such packages and their features:

- **dotenv:** For secure configuration management.
- **nodemon:** A utility for automatic server restarts.

This is just an overview of these packages. Each deserves in-depth exploration to fully grasp its capabilities and best practices.

Dotenv: Secure configuration management

Dotenv enables us to store and manage sensitive information, like API keys, in a `.env` file—a simple text file used to define environment variables as key-value pairs. Using a `.env` file keeps sensitive data out of source code, but the file itself must still be protected, excluded from version control (e.g. via `.gitignore`), and never committed or shared. To ensure it remains secure, it should be excluded from version control (e.g., by adding it to `.gitignore`). The following example uses the `dotenv` package to load environment variables defined in the `.env` file.



Nodemon: Enhancing development workflow

Nodemon automatically restarts the Node.js application whenever we make changes to the code, speeding up the development process. It can be installed globally or, more commonly in modern workflows, as a local devDependency (`npm install --save-dev nodemon`) and run via `npx nodemon` or an npm script. Let's test Nodemon with a simple HTTP server.

Note: Follow the steps given below to start the server:

- Install nodemon globally using the `npm install -g nodemon` command.
- Run the server using nodemon with the `nodemon index.js` command.

This will start the server successfully showing the message "Server running on port 3000".



A screenshot of a code editor with a file explorer on the left and a code editor on the right. The file explorer shows two files: `updated-index.js` and `index.js`. The code editor is open to `index.js` and contains the following code:

```
1 const http = require('http'); // Import HTTP module
2 const port = 3000; // Define server port
3
4 // Create a basic HTTP server
5 const server = http.createServer((req, res) => {
6   res.end('Hello, Nodemon!'); // Send response
7 });
8
9 // Start the server
10 server.listen(port, () => {
11   console.log('Server running on port ${port}');
12 });
```



A screenshot of a code editor with a file explorer on the left and a code editor on the right. The file explorer shows two files: `updated-index.js` and `index.js`. The code editor is open to `updated-index.js` and contains the following code:

```
1 const http = require('http'); // Import HTTP module
2 const port = 3000; // Define server port
3
4 // Create a basic HTTP server
5 const server = http.createServer((req, res) => {
6   res.end('Hello, Nodemon!'); // Send response
7 });
8
9 // Start the server
10 server.listen(port, () => {
11   console.log('HTTP Server is live at http://localhost:${port}');
12 });
```

Use nodemon for automatic server restarts during development

Test the server

When the server is running, open a new terminal by clicking the “+” tab, then navigate to the project directory and replace the `index.js` file using the following commands:

```
cd /usercode/my-project
```

```
cp updated-index.js index.js
```

Switch to the terminal tab running the server and observe the server logs as nodemon detects the changes and restarts the server. The updated file includes a different message, and we should see the following output in the terminal:

[nodemon] restarting due to changes...

[nodemon] starting `node index.js`

Server running on port 3000

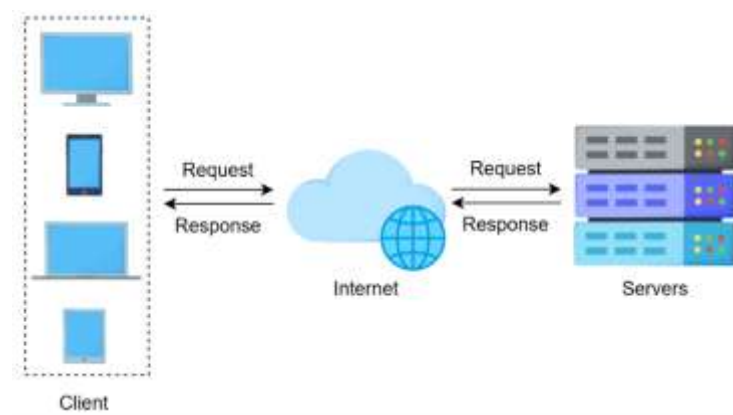
Explanation

1. **Line 1:** Import the `http` module to create an HTTP server.
2. **Line 2:** Define the port number for the server.
3. **Line 5:** Create an HTTP server and handle requests with a callback function.
4. **Line 6:** Send a text response ("Hello, Nodemon!") to the client.
5. **Lines 10–12:** Start the server and log a confirmation message to the console.

The two files, `index.js` and `updated-index.js`, follow the same structure, with the only difference being in **line 11**, where the console log message has been updated.

Building a Basic Web Server

Creating a Simple HTTP Server



Client-server communication over the internet

Overview of HTTP requests and responses

When a client (like a browser or app) interacts with a web server, it sends an HTTP request to the server, which processes the request and responds with an HTTP response. Think of the request as a message asking the server for information (like a web page, data, or resource) and the response as the server's reply containing the requested content.

A simple HTTP request typically includes:

- **URL:** This is the path or address being requested on the server, such as `/` (the homepage) or `/about` (a specific page). The URL helps the server determine what content or functionality the client is asking for. For example, `/products` might return a list of items, while `/contact` could display a contact form.
- **Method:** The HTTP method specifies the type of action the client wants to perform. Common methods include:
 - **GET:** Used to retrieve information from the server (e.g., loading a webpage or fetching data from an API).

- **POST:** Used to send data to the server, often for creating or updating resources (e.g., submitting a form or uploading a file).

The server then returns a response, which includes:

- **Status code:** A code indicating the status of the request (e.g., 200 for success).
- **Content:** The data the client requested, such as HTML or JSON.

Setting up the server with the `http` module

Node.js provides the built-in `http` module, which enables us to create a server, define its behavior, and listen for incoming requests.

Here are the steps to create a simple server:

1. Import the `http` module.
2. Use `http.createServer()` to create a server instance.
3. Define how the server should handle requests.
4. Use `server.listen()` to specify the port on which the server should listen

Basic Node.js server that responds with "Hello, World!" to any request.

```
const http = require('http'); // Import the http module
// Create the server
const server = http.createServer((req, res) => {
  res.write('Hello, World!'); // Write a response to the client
  res.end(); // End and send the response
});

// Listen on port 3000
server.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Explanation

- **Line 1:** Imports the built-in `http` module, which provides essential functionality for creating HTTP servers in Node.js.
- **Line 4:** Creates an HTTP server instance using `http.createServer()`. This function takes a callback with `req` (request) and `res` (response) parameters, representing the client's request and the server's response.
- **Line 5:** Writes the string "Hello, World!" as the response content to be sent to the client.
- **Line 6:** Ends the response, ensuring that the client receives the complete message. Without this, the client would wait for more data indefinitely.
- **Line 10:** Starts the server, instructing it to listen for incoming requests on port 3000.
- **Line 11:** Logs a message to the console to confirm the server is running and ready to receive requests on <http://localhost:3000>.

Understanding the `req` and `res` objects

In the server code, two main objects handle the interaction between the client and the server. Both of these objects are streams, allowing efficient handling of data in chunks:

- **`req` (Request object):** Represents the client's HTTP request, containing information about what the client wants. It is an instance of a Readable Stream, which means it can emit data in chunks as it arrives. Key properties include:
 - `req.url`: The URL path requested by the client (e.g., `/about`).
 - `req.method`: The HTTP method used, such as GET or POST.
 - We can also use `req.on('data')` and `req.on('end')` to process incoming data (e.g., for processing a POST body).
- **`res` (Response object):** Represents the server's response, used to send data back to the client. It is an instance of a Writable Stream, enabling the server to send the response in chunks. Key methods include:
 - `res.writeHead()`: Sets the HTTP status code and response headers, such as `Content-Type`.
 - `res.write()`: Writes data to the response body, which can be sent in multiple chunks if needed.
 - `res.end()`: Completes the response and sends it to the client.

These objects allow us to control the server's response, enabling us to respond differently based on the client's request.

Basic Routing

In web development, **routing** refers to defining how a server handles incoming client requests based on the requested URL and HTTP method. It serves as a blueprint for directing requests to the appropriate responses, enabling the server to serve different purposes dynamically. For example, routing can be used to display a homepage, retrieve or submit data, handle user authentication, or show an error message for unknown routes.

By implementing routing, a server can map specific URLs or request patterns to dedicated logic, making it flexible and capable of handling various client needs. This approach is foundational for building interactive web applications and APIs, as it allows developers to create tailored responses for different endpoints, such as `/about`, `/api/data`, or `/404`.

Node.js can efficiently handle many incoming requests concurrently, while your routing logic decides how each one is handled. In this lesson, we'll create a basic server that serves different responses depending on the requested URL.

How routing works in Node.js

This lesson demonstrates basic, exact-match routing based on `req.url` and `req.method`. Full-featured frameworks typically add support for URL parameters, query strings, and wildcards on top of this. By checking `req.url` and `req.method`, we can define routes to handle specific requests:

- `/` can serve as the homepage.
- `/about` can provide information about the application.
- Additional routes can be added to serve data or perform specific actions.

This enables us to build a flexible server that serves different content depending on the client's request.

Setting up routes with the `http` module

Let's add routing to our server to respond differently to each URL. To add basic routing:

1. Import the `http` module.
2. Create a server using `http.createServer()`.
3. Check the `req.url` and `req.method` properties inside the server callback.
4. Use `res.write()` and `res.end()` to send different responses based on the requested route.

Example:

```
const http = require('http'); // Import the http module
// Create the server with routing logic
const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.write('Welcome to the Homepage!');
  } else if (req.url === '/about' && req.method === 'GET') {
    res.write('About this server');
  } else {
    res.statusCode = 404;
    res.write('404 Not Found');
  }
  res.end();
});
// Listen on port 3000
server.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Explanation

- **Line 1:** Imports the built-in `http` module.
- **Lines 4–13:** Creates an HTTP server that checks both `req.url` (the URL path) and `req.method` (HTTP method) to determine how to respond.
 - `req.url === '/' && req.method === 'GET'`: If the request is for the homepage (`/`), the server responds with "Welcome to the Homepage!".
 - `req.url === '/about' && req.method === 'GET'`: If the request is for `/about`, the server responds with "About this server".

- **404 case:** For any other URL, the server sets the status code to **404** and responds with the text **404 Not Found**.
- The **res.end()** method completes the response.
- **Lines 17–19:** Starts the server on port **3000** and logs the URL to the console.

Serving HTML and JSON Responses

In modern applications, servers need to handle diverse content types. For example, HTML is used for rendering web pages, and JSON is used for exchanging structured data, commonly in APIs. To handle these use cases, servers must include headers to specify the format of their responses.

The **Content-Type** header in HTTP responses is particularly important as it tells clients (like browsers or API consumers) how to interpret the returned data. For example:

- **text/html** specifies HTML content for rendering web pages.
- **application/json** denotes structured data formatted as JSON.
- **text/plain** indicates plain text.

Setting the correct **Content-Type** helps clients correctly interpret the response and is the standard way to describe the payload format.

Building a server for HTML and JSON responses

Let's create a server that responds with HTML content for one route and JSON data for another. To set the appropriate headers for each response, we'll use the **res.writeHead** method. This method allows us to define both the HTTP status code and headers for the response.

Note: Clicking the provided URL will open the **/html** endpoint by default in a new browser tab, displaying the HTML response. To test the **/json** endpoint, replace **/html** with **/json** in the URL.

```
const http = require('http');
//A Node.js server that serves HTML for /html and JSON for /json routes
const server = http.createServer((req, res) => {
  if (req.url === '/html') {
    // Respond with HTML
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.write('<h1>Welcome to Node.js</h1><p>This is an HTML response.</p>');
  } else if (req.url === '/json') {
    // Respond with JSON
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.write(JSON.stringify({ message: 'Hello, this is JSON!' }));
  } else {
    // Respond with 404 for other routes
    res.statusCode = 404;
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.write('404 Not Found');
  }
  res.end();
});

server.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Explanation

- **Line 1:** Imports the `http` module for creating the server.
- **Line 3:** Creates a server that listens for requests.
- **Lines 4–7:** Checks if the URL is `/html`. If true, sets the `Content-Type` to `text/html` using `res.writeHead` and writes an HTML response.
- **Lines 8–11:** Checks if the URL is `/json`. If true, sets the `Content-Type` to `application/json` and writes a JSON response.
- **Lines 12–18:** For any other URL, returns a "404 Not Found" response with plain text content.
- **Lines 21–23:** Starts the server on port `3000`.

Understanding status codes in `writeHead`

The first parameter of `res.writeHead` specifies the HTTP status code. These codes provide information to the client about the outcome of its request. For instance:

- **200 OK:** The request was successful, and the response contains the requested data.
- **404 Not Found:** Tells the client the requested resource could not be found. The client may display its own error page or use the status code programmatically; exact behavior depends on the client.
- **500 Internal Server Error:** An error occurred while processing the request.

Setting status codes ensures that the client understands not just the content of the response but also its context. For example:

- A **200** status code allows browsers or API consumers to process and display the content.
- **404** status code prompts browsers to show an error message.

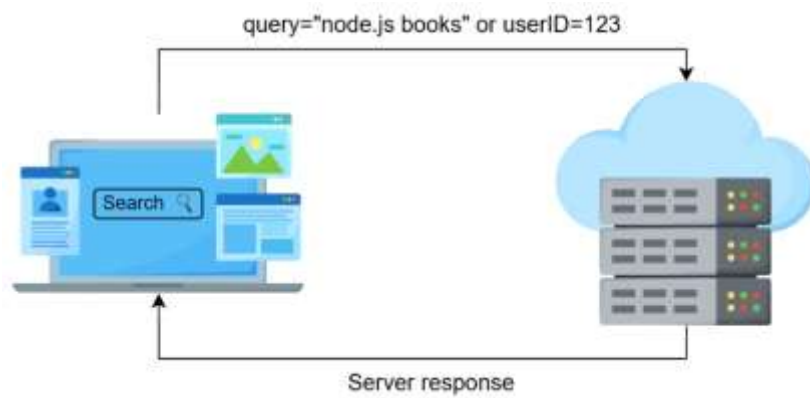
By using `writeHead` to specify these codes, we make the server's behavior more predictable and compliant with HTTP standards.

Handling GET and POST Parameters

In web development, servers often need more information from clients to provide meaningful responses. **Parameters** are the pieces of data that clients send to servers to specify exactly what they need. By handling parameters effectively in our Node.js server, we can create dynamic applications that respond intelligently to user input.

Think about when we search for a product online or view a specific user's profile:

- When we search, our query (e.g., "node.js books") is sent to the server as a parameter, so it knows what results to return.
- When we view a profile, the user ID in the URL tells the server which user's information to display.
- When we submit a form, the data we enter is sent as parameters for the server to process.



Server processing parameters to deliver relevant responses

Client input can be conveyed in different forms:

- **Query strings:** Key-value pairs appended to the URL, like `/search?term=nodejs`.
- **Path parameters:** Dynamic segments of the URL, such as `/user/123`, where `123` is a parameter indicating a specific user.
- **POST body data:** Data sent within the body of a POST request. This isn't itself a "parameter" until it's parsed into named fields. For example, form fields via `URLSearchParams` or JSON via `JSON.parse`.

By mastering how to handle these parameters in Node.js, we'll be able to create dynamic, interactive web applications that respond to user input in meaningful ways.

Understanding GET and POST methods

Now that we've explored how parameters enable dynamic interactions between clients and servers, let's explore how these parameters are transmitted using different HTTP methods. Among the most widely used methods are GET and POST, which we introduced earlier. Each serves a distinct purpose in web communication.

Understanding when to use each method is essential for building robust applications. Each has its own strengths and appropriate scenarios:

- GET is ideal for requests where data retrieval does not change the server's state.
- POST is suitable when the request results in a change on the server, such as updating a database.

While GET and POST are the most commonly used HTTP methods and are central to most web interactions, other methods like PUT, DELETE, and PATCH also play specific roles in managing server resources. We'll focus on GET and POST here, as they form the foundation of client-server communication.

	GET method	POST method
Purpose	Retrieve data from a server.	Send data to a server, often to create or update resources
Parameter transmission	Parameters are included in the URL as query strings.	Data is sent within the request body, not visible in the URL. This becomes usable parameters once parsed.
Typical use cases	Fetching or filtering data, such as retrieving a webpage or searching for information.	Submitting form data, uploading files, or handling sensitive information.

Handling GET parameters

GET parameters include query strings. URLs can also carry path parameters (like `/user/123`), but these aren't a built-in HTTP feature. They're a convention you implement yourself or get from a framework. These provide a way to send information in the URL.

Query strings

Query strings allow clients to pass data in the URL. For example, `/search?term=nodejs` contains the query parameter `term` with the value `nodejs`.

A basic server that extracts and responds to query string parameters

```
const http = require('http');
const { URL } = require('url');

const server = http.createServer((req, res) => {
  const parsedUrl = new URL(req.url, `http://${req.headers.host}`);

  if (parsedUrl.pathname === '/search' && req.method === 'GET') {
    const term = parsedUrl.searchParams.get('term') || 'No term provided';

    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.write(`Search results for: ${term}`);
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.write('404 Not Found');
  }
  res.end();
});

server.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Testing the server

Once the server is running, click the “+” tab to open a new terminal and use the following `curl` commands to test it:

1. With a search term:

```
2. curl "http://localhost:3000/search?term=nodejs"
```

Expected output:

```
Search results for: nodejs
```

3. Without a search term:

```
4. curl "http://localhost:3000/search?"
```

Expected output:

```
Search results for: No term provided
```

Explanation

1. **Line 2:** Imports the `URL` class from Node's built-in `url` module for reliable URL parsing.
2. **Line 4:** A server is created using `http.createServer()`, and the callback function is used to handle incoming requests.
3. **Line 5:** Parses the request URL using `new URL()`, giving access to `pathname` and `searchParams`.
4. **Line 7:** Checks if the `pathname` is `/search` and the method is GET.
5. **Line 8:** Extracts the `term` query parameter using `searchParams.get('term')`, defaulting to "No term provided" if absent.
6. **Lines 10–11:** Responds with the search term as plain text.
7. **Lines 13–14:** Returns a 404 response for any other route.
8. **Line 19:** The server starts listening on port `3000`.

Path parameters

Path parameters allow us to define flexible, dynamic routes by using placeholders in the URL. For example, `/user/123` can represent a route where `123` is a parameter identifying a specific user. In plain Node.js there's no built-in router, so we extract this manually using a regular expression; frameworks like Express handle this for you automatically. This enables the server to handle similar requests with reusable logic, tailoring responses based on the parameter's value. Let's look at an example:

A server that uses regular expressions to extract and respond to path parameters

```
const http = require('http');
const server = http.createServer((req, res) => {
  const match = req.url.match(/^\/user\/(\d+)$/); // Matches /user/:id where :id is a number
  if (match && req.method === 'GET') {
    const userId = match[1]; // Extract the user ID
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.write(`User ID: ${userId}`);
  }
});
```

```
    } else {  
      res.writeHead(404, { 'Content-Type': 'text/plain' });  
      res.write('404 Not Found');  
    }  
    res.end();  
  });  
  server.listen(3006, () => console.log('Server running on http://localhost:3006'));
```

Testing the server

Once the server is running, click the “+” tab to open a new terminal and use the following `curl` commands to test it:

1. Testing with a valid user ID:

```
2. curl "http://localhost:3000/user/123"
```

Expected output:

```
User ID: 123
```

3. Testing with an invalid user ID:

```
4. curl "http://localhost:3000/user/abc"
```

Expected output:

```
404 Not Found
```

Explanation

1. **Line 4:** A regular expression matches the URL pattern `/user/:id` where `:id` is a number.
2. **Lines 6–7:** If the URL matches and the method is GET, the `userId` is extracted from the captured group in the regular expression.
3. **Lines 8–9:** Responds with the extracted user ID.
4. **Lines 10–13:** If the route does not match, the server responds with a 404 error.

Handling POST parameters

POST parameters are sent in the request body and are commonly used for transmitting form data or JSON payloads. In Node.js, the `req` object, which is a Readable Stream, allows us to process this data efficiently.

When data is sent in chunks (as is typical with POST requests), the `req` object emits specific events to signal its progress:

- **data:** Triggered whenever a chunk of data is received. This allows the server to process incoming data incrementally.
- **end:** Triggered when all chunks have been received, indicating that the full body is ready for processing.

We've previously explored the role of streams in handling data efficiently. Here, we'll focus on how these events enable us to handle POST data effectively.

A server that processes POST body data and sends a JSON response

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.url === '/submit' && req.method === 'POST') {
    let body = "";
    // Collect the request body
    req.on('data', chunk => {
      body += chunk.toString();
    });
    // Handle errors on the request stream
    req.on('error', () => {
      res.writeHead(400, { 'Content-Type': 'text/plain' });
      res.end('Bad Request');
    });
    // Process the body when fully received
    req.on('end', () => {
      const params = new URLSearchParams(body);
      const name = params.get('name') || 'Anonymous';
      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ name }));
    });
  } else {
    // Handle other routes
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 Not Found');
  }
});
server.listen(3000, () => console.log('Server running on http://localhost:3000'));
```

Testing the server

Once the server is running, click the “+” tab to open a new terminal and use the following `curl` commands to test it:

1. Testing with a sample string:

```
2. curl -X POST -d "name=inam" "http://localhost:3000/submit"
```

Expected output:

```
{"name":"inam"}
```

3. Testing with another sample string:

```
4. curl -X POST -d "name=Node.js" "http://localhost:3000/submit"
```

Expected output:

```
{"name":"Node.js"}
```

Explanation

- **Line 4:** The route `/submit` is checked, and the HTTP method must be POST to proceed.
- **Lines 6–9:** The `data` event listener collects chunks of the incoming request body. Each chunk is converted to a string and appended to the body variable.
- **Lines 10–14:** The `error` event listener handles problems on the request stream, responding with a 400 status and "Bad Request" if triggered.
- **Lines 15–21:** The `end` event listener triggers once all chunks have been received, ensuring the entire body is available for processing.
 - **Lines 16–20:** The collected `body` string is parsed with `URLSearchParams` to extract the `name` field, defaulting to "Anonymous" if not provided. The result is sent back as a JSON response, with `Content-Type` set to `application/json`.
- **Lines 22–26:** For any other route or method, the server responds with a "404 Not Found" error and a `text/plain` content type.
- **Line 28:** The server is started on port `3000`, and a log message confirms it is running.

Connecting to MongoDB and Performing CRUD Operations in Node.js

1. Connecting to MongoDB via Node.js

MongoDB is a NoSQL database that stores data as JSON-like documents (BSON internally). To talk to it from Node.js, we use the official `mongodb` driver package.

Installing the driver

```
npm init -y
npm install mongodb
```

Steps to connect

- Import `MongoClient` from the `mongodb` package.
- Define the connection string (URI) for your MongoDB instance.
- Create a `MongoClient` instance with that URI.
- Call `client.connect()` to open the connection.
- Select a database and collection to work with.
- Always close the connection when done (or keep it open for the lifetime of a server).

Example: Basic connection

Basic Node.js script that connects to a local MongoDB instance.

```
const { MongoClient } = require('mongodb');
// Connection URI - local MongoDB instance
const uri = 'mongodb://127.0.0.1:27017';
// Create a new client
const client = new MongoClient(uri);
async function connectDB() {
  try {
    await client.connect(); // Establish connection
    console.log('Connected to MongoDB successfully!');
    const db = client.db('myDatabase'); // Select (or create) a database
    console.log('Using database:', db.databaseName);
  } catch (err) {
    console.error('Connection error:', err);
  } finally {
    await client.close(); // Close connection when done
  }
}
connectDB();
```

Explanation

- **MongoClient:** The class used to connect to a MongoDB server.
- **uri:** The connection string. For MongoDB Atlas (cloud), it looks like `mongodb+srv://<user>:<password>@cluster.mongodb.net/`.
- **client.connect():** Returns a Promise; must be awaited before using the client.
- **client.db('myDatabase'):** MongoDB creates the database lazily — it only actually appears once you insert data into it.
- **client.close():** Releases the connection. In a long-running server, you'd typically keep the connection open instead of closing it after every operation.

Testing the connection

Make sure MongoDB is running locally (`mongod`), then run:

```
node connect.js
```

Expected output:

```
Connected to MongoDB successfully!
Using database: myDatabase
```

2. Connecting and Performing CRUD Operations

CRUD stands for Create, Read, Update, Delete — the four basic operations you perform on data. Once connected, we get a reference to a collection (similar to a table in relational databases) and call methods on it.

Setting up a reusable connection

```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://127.0.0.1:27017';
const client = new MongoClient(uri);
let usersCollection;
async function initDB() {
  await client.connect();
  const db = client.db('myDatabase');
  usersCollection = db.collection('users');
  console.log('Database connected, collection ready.');
```

```
}
module.exports = { initDB, client, getCollection: () => usersCollection };
```

Create — inserting documents

```
async function createUser() {
  const result = await usersCollection.insertOne({
    name: 'Inam',
    age: 25,
    email: 'inam@example.com'
  });
  console.log('Inserted document ID:', result.insertedId);
}
```

- `insertOne()`: Inserts a single document; returns an object containing the generated `_id`.
- `insertMany([...])`: Inserts an array of documents in one call — more efficient than looping `insertOne()`.

Read — retrieving documents

```
async function readUsers() {
  // Find all documents
  const allUsers = await usersCollection.find({}).toArray();
  console.log('All users:', allUsers);

  // Find one document matching a filter
  const oneUser = await usersCollection.findOne({ name: 'Inam' });
  console.log('Found user:', oneUser);

  // Find with a condition (age greater than 20)
  const filtered = await usersCollection.find({ age: { $gt: 20 } }).toArray();
  console.log('Users older than 20:', filtered);
}
```

- `find(filter)`: Returns a cursor; `.toArray()` converts it into a usable array.
- `findOne(filter)`: Returns the first matching document, or `null` if none found.
- **Query operators** like `$gt`, `$lt`, `$in` let you build conditions similar to SQL's WHERE clause.

Update — modifying documents

```
async function updateUser() {
  const result = await usersCollection.updateOne(
    { name: 'Inam' },           // filter: which document to match
    { $set: { age: 26 } }      // update: what to change
  );
  console.log('Matched:', result.matchedCount, 'Modified:',
    result.modifiedCount);
}
```

- `updateOne(filter, update)`: Updates the first document matching the filter.
- `updateMany(filter, update)`: Updates all matching documents.
- `$set`: An update operator that sets specific fields without overwriting the whole document.

Delete — removing documents

```
async function deleteUser() {
  const result = await usersCollection.deleteOne({ name: 'Inam' });
  console.log('Deleted count:', result.deletedCount);
}
```

- `deleteOne(filter)`: Deletes the first matching document.
- `deleteMany(filter)`: Deletes all matching documents.

Putting it together

```
const { MongoClient } = require('mongodb');
const uri = 'mongodb://127.0.0.1:27017';
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const db = client.db('myDatabase');
    const users = db.collection('users');

    // Create
    await users.insertOne({ name: 'Inam', age: 25 });
    // Read
    const found = await users.findOne({ name: 'Inam' });
    console.log('Found:', found);
    // Update
    await users.updateOne({ name: 'Inam' }, { $set: { age: 26 } });
    // Delete
    await users.deleteOne({ name: 'Inam' });
    console.log('CRUD operations completed.');
```

Connecting CRUD to an HTTP server (tying it back to Part 1)

Just like the routing examples in the earlier document, you can wire these CRUD functions into `http.createServer()` routes:

```
const http = require('http');
const { MongoClient, ObjectId } = require('mongodb');

const client = new MongoClient('mongodb://127.0.0.1:27017');
let users;

// Helper to parse request body
function getRequestBody(req) {
  return new Promise((resolve, reject) => {
    let body = '';
    req.on('data', chunk => (body += chunk));
    req.on('end', () => {
      try {
        resolve(body ? JSON.parse(body) : {});
      } catch (err) {
        reject(err);
      }
    });
  });
}

// Helper to send JSON response
function sendJSON(res, statusCode, data) {
  res.writeHead(statusCode, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(data));
}

const server = http.createServer(async (req, res) => {
  const url = req.url;
  const method = req.method;

  try {
    // ----- CREATE -----
    if (url === '/users' && method === 'POST') {
      const data = await getRequestBody(req);
      const result = await users.insertOne(data);
      sendJSON(res, 201, { insertedId: result.insertedId });

      // ----- READ (all) -----
    } else if (url === '/users' && method === 'GET') {
      const allUsers = await users.find({}).toArray();
      sendJSON(res, 200, allUsers);

      // ----- READ (single, by ID) -----
    } else if (url.match(/^\/users\/[a-fd]{24}$/) && method === 'GET') {
      const id = url.split('/')[2];
      const user = await users.findOne({ _id: new ObjectId(id) });
      if (!user) return sendJSON(res, 404, { error: 'User not found' });
    }
  }
});
```

```

    sendJSON(res, 200, user);

// ----- UPDATE -----
} else if (url.match(/^\/users\/[a-f\d]{24}$/) && method === 'PUT') {
  const id = url.split('/')[2];
  const data = await getRequestBody(req);
  const result = await users.updateOne(
    { _id: new ObjectId(id) },
    { $set: data }
  );
  if (result.matchedCount === 0) return sendJSON(res, 404, { error: 'User not found' });
  sendJSON(res, 200, { modifiedCount: result.modifiedCount });

// ----- DELETE -----
} else if (url.match(/^\/users\/[a-f\d]{24}$/) && method === 'DELETE') {
  const id = url.split('/')[2];
  const result = await users.deleteOne({ _id: new ObjectId(id) });
  if (result.deletedCount === 0) return sendJSON(res, 404, { error: 'User not found' });
  sendJSON(res, 200, { message: 'User deleted successfully' });

// ----- 404 -----
} else {
  sendJSON(res, 404, { error: 'Route not found' });
}
} catch (err) {
  sendJSON(res, 500, { error: err.message });
}
});

client.connect().then(() => {
  users = client.db('myDatabase').collection('users');
  server.listen(3000, () => console.log('Server running on http://localhost:3000'));
}).catch(err => console.error('MongoDB connection failed:', err));

```

Testing with curl

```

# CREATE
curl -X POST -H "Content-Type: application/json" \
  -d '{"name":"Node.js","age":10}' \
  http://localhost:3000/users
# READ all
curl http://localhost:3000/users
# READ one (replace <id> with an actual _id from a previous response)
curl http://localhost:3000/users/<id>
# UPDATE
curl -X PUT -H "Content-Type: application/json" \
  -d '{"age":11}' \
  http://localhost:3000/users/<id>
# DELETE
curl -X DELETE http://localhost:3000/users/<id>

```